

烟幕干扰弹的投放策略

摘 要

本文围绕无人机投放烟幕干扰弹保护真目标、干扰来袭导弹的核心需求，针对五类投放场景构建模型与优化策略。基于忽略空气阻力等假设，通过运动学方程建立各实体轨迹模型，以烟幕云团到导弹-目标连线距离 ≤ 10 米为有效遮蔽标准。采用遗传算法等智能算法，结合覆盖矩阵、混合整数规划等方法，求解不同场景下无人机航向、速度、投放时刻等最优参数，通过多维度可视化验证效果并保存结果。实现从简单到复杂场景的拓展，为烟幕投放策略制定提供科学依据，最大化有效遮蔽时长。

针对问题一 针对 FY1 投 1 枚烟幕弹干扰 M1，本文建三维坐标系，用运动学方程构建导弹、无人机、烟幕弹轨迹模型。得到有效遮蔽时长 1.41 秒，为后续优化提供基准模型。

针对问题二 FY1 投 1 枚烟幕弹最大化遮蔽时长，本文以航向角、速度、投放时刻、起爆延迟为优化变量，最优参数对应最大有效遮蔽时长 8.72 秒，显著提升

针对问题三 FY1 投 3 枚烟幕弹干扰 M1，本文用“候选动作离散化+覆盖矩阵”法，将投放时刻、起爆延迟视为候选动作，构混合整数规划模型，加投放间隔约束。

针对问题四 用遗传算法等寻优，三机通过方向与时间协同，形成互补遮蔽区间，最佳遮蔽时长 7.12 秒，实现多无人机单弹协同效果提升，为多平台协同提供思路。

针对问题五 本文构建大规模混合整数规划模型，预计算动作遮蔽贡献张量，用遗传算法结合剪枝策略寻优。最优策略总有效遮蔽时长 68.37 秒，无人机多方向形成立体烟幕网，实现多平台、多弹、多目标全局最优遮蔽。

关键词：粒子群优化算法、遗传算法、混合整数规划、覆盖矩阵

一、问题背景与重述

1.1 问题背景

烟幕干扰弹主要通过化学燃烧或爆炸分散形成烟幕或气溶胶云团，在目标前方特定空域形成遮蔽，干扰敌方导弹，具有成本低等特点。随着烟幕干扰技术的不断发展，现已有多种投放方式完成烟幕干扰弹的定点精确抛撒，即在抛撒前能精确控制烟幕干扰弹到达预定位置，通过时间引信时序控制起爆时间。现考虑运用无人机完成烟幕干扰弹的投放策略问题，利用无人机投放烟幕干扰弹在来袭武器和保护目标之间形成烟幕遮蔽。

1.2 问题重述

烟幕干扰弹投放策略问题背景

烟幕干扰弹投放策略的核心目标，是通过无人机投放烟幕干扰弹，在来袭空地导弹与真目标间形成有效遮蔽，避免真目标被导弹发现，并设计策略使遮蔽时间得到延长。烟幕干扰弹起爆后，球状云团会随之形成，云团会匀速下沉，且仅在特定范围及时长内可实现有效遮蔽；无人机受领任务后，飞行方向可被瞬时调整，无人机将按特定速度匀速飞行，干扰弹投放时的时间间隔要求需被遵守。真目标旁设有假目标，假目标会被来袭导弹直指。雷达发现导弹后，任务会被控制中心立即指派给多架无人机，需针对单架投单枚或多枚、多架协同投放等场景，确定飞行及干扰弹参数，以实现真目标的高效遮蔽。

问题一：利用无人机 FY1 投放 1 枚烟幕干扰弹实施对 M1 的干扰，若 FY1 以 120 m/s 的速度朝向假目标方向飞行，受领任务 1.5 s 后即投放 1 枚烟幕干扰弹，间隔 3.6 s 后起爆。请给出烟幕干扰弹对 M1 的有效遮蔽时长。

问题二：利用无人机 FY1 投放 1 枚烟幕干扰弹实施对 M1 的干扰，确定 FY1 的飞行方向、飞行速度、烟幕干扰弹投放点、烟幕干扰弹起爆点，使得遮蔽时间尽可能长。

问题三：利用无人机 FY1 投放 3 枚烟幕干扰弹，实施对 M1 的干扰。请给出烟幕干扰弹的投放策略，并将结果保存到文件 result1.xlsx 中（模板文件见附件）。

问题四：利用 FY1、FY2、FY3 等 3 架无人机，各投放 1 枚烟幕干扰弹，实施对 M1 的干扰。请给出烟幕干扰弹的投放策略，并将结果保存到文件 result2.xlsx 中（模板文件见附件）。

问题五：利用 5 架无人机，每架无人机至多投放 3 枚烟幕干扰弹，实施对 M1、M2、M3 等 3 枚来袭导弹的干扰。请给出烟幕干扰弹的投放策略，并将结果保存到文件 result3.xlsx 中（模板文件见附件）。

二、模型假设

- 1: 忽略空气阻力和风的影响。
- 2: 无人机高度恒定。
- 3: 烟幕弹初速等于无人机投放瞬间速度。
- 4: 云团起爆瞬时成球，半径恒为 R，仅在有效时间窗内有效。

三. 符号说明

符号	说明	符号	说明
v_M	导弹速度	$\mathcal{Q}_{j,k}$	由无人机 r_{I_j} 投下的第 k 枚烟幕弹
K_c	云团有效半径	$t_{h,j}$	目标时间
v_c	云团下沉速率	φ_j	航向角
M_i	来袭导弹的第 i 枚	r_{jk}	云团爆炸位置
r_{I_j}	第 j 架无人机	$c_{j,k}$	云团起爆中心轨迹
$L_i(s,t)$	视线段	$s(t)$	最近点
r_M	位置坐标	λ	时间网格
$d(\phi)$	单位航向向量	ρ	投放时刻
D	起爆延时候选	x_k	决策变量
κ	动作集合	ξ	机动模式集合
A	覆盖矩阵	$J_{jh}(t)$	无人机轨迹
σ_x	软指示函数	$s_{ijk}(t)$	遮蔽软指示函数
ζ	同机冲突集合	s_{ink}	覆盖张量
n_{jh}	航迹松弛变量	r_{ijn}	单机聚合覆盖指标
z_m	全局变量	T	总遮蔽时间

四. 问题分析

4.1 问题一的分析

由文献[1]得知，导弹、无人机和烟幕弹的初始条件下，先确定三者和时间轴上的运动轨迹在导弹沿直线匀速飞向目标，无人机按照给定的速度和航向移动，烟幕弹在投放后延时起爆并形成云团。接着，需要比较导弹飞行轨迹与云团的相对位置关系，当导弹到目标的视线经过云团且处于云团的有效时间范围内时，视为形成遮蔽。通过这一过程统计导弹被遮蔽的时间段，并合并得到总的遮蔽时长。

4.2 问题二的分析

在单枚烟幕弹的情况下，问题转化为优化无人机的飞行参数与投放时机。具体而言，将航向、速度、投放时刻和起爆延时作为待选变量，在满足速度范围、延时和任务约束的前提下，分别模拟不同参数组合的遮蔽效果。通过对比不同方案下的遮蔽时长，找到能最大化遮蔽时间的组合，从而确定最优的飞行与投放策略。

4.3 问题三的分析

由文献[4]得知，当无人机投放三枚烟幕弹时，需要协调三次投放的时间与空间位置。可以将每一次投放视作一个动作，每个动作会带来一个在时空中分布的云团遮蔽效应。问题的关键在于合理安排三次投放的顺序和间隔，使多个云团在时间上尽量衔接，避免同时重叠浪费，从而延长整体的遮蔽覆盖。最终在满足投放间隔限制的条件下，选出最佳的投放时刻和对应的遮蔽时段

4.4 问题四的分析

在三架无人机协同的情况下，需要为每架无人机选择一次投放方案。三架无人机的动作相互独立，但对导弹的遮蔽效应会叠加。因此，问题的核心是为每架无人机分别选择合适的投放时刻与参数，使得三次云团遮蔽在时间轴上能够形成较好的接力与互补，从而获得最长的总遮蔽时间。最终通过比较不同组合，确定三机配合下的最优投放策略。

4.5 问题五的分析

当引入五架无人机和三枚导弹时，问题变为大规模的多目标优化。每架无人机可以投放最多三次，因此会产生大量候选动作，每个动作对三枚导弹的遮蔽效果不同。需要在满足无人机投放次数和时间间隔限制的条件下，综合选择动作组合，使三枚导弹的总体遮蔽时间最长。为此，需要考虑不同无人机在时间上的错

峰配合，以及在多导弹之间的合理分配，以避免资源浪费并提升整体覆盖效果。最终输出能兼顾全局最优的多无人机协同方案。

五、模型的建立与求解

5.1 问题一模型的建立与求解

根据本小组的基本假设，建立以假目标为原点的三维坐标系以描述导弹、无人机与烟幕弹的位置随时间的变化；再分别用运动学方法确定导弹的匀速直线逼近轨迹、无人机按给定航向与速度的直线匀速轨迹以及烟幕弹在脱离后的自由落体轨迹与起爆后云团中心的下沉轨迹；然后根据云团有效时间窗与空间半径，采用最近点距离或时间步进检查导弹到目标的连线是否穿过云团并位于云团有效时段，从而判定每一时间是否被遮蔽；最后把所有被判定为“遮蔽”的时刻合并计时就可以得到总有效遮蔽时长。

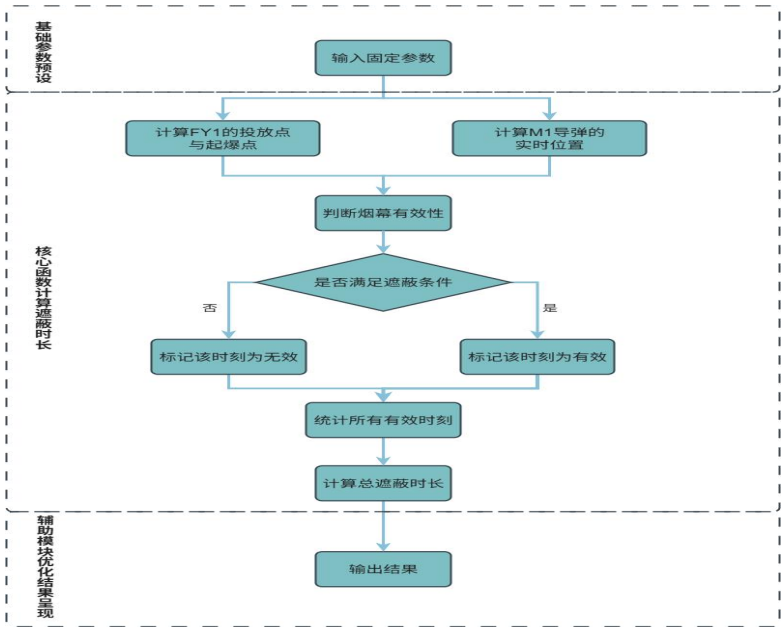


图 1 问题一流程图

5.1.1 FY1 投放一枚烟幕弹干扰 M1 的模型

(1) 导弹、无人机、烟幕弹运动学建模

先确立坐标系，以目标点为原点 $O(0,0,0)$ ，水平方向为 xy 平面，竖直方向为 z 轴，（向上为正）真目标几何中心位于 $c=(0,200,5)$ ，为圆柱体，半径 $R>0$ ，高

度 $H>0$ 。重力加速 g 取 9.783 m/s^2 。

①导弹 M1 速度微分方程的建立

速度方向始终指向原点

$$\frac{d_{r_{Mi}}}{dt} = -v_M \frac{r_{Mi}(t)}{\|r_{Mi}(t)\|} \quad (1)$$

其中, $r_{Mi}(0)$ 命中, 目标时间为

$$t_{h,i} = \frac{\|r_{Mi}(0)\|}{v_M}, t \in [0, t_{h,i}] \quad (2)$$

②无人机 FY_j 速度方程的) 建立

$$\frac{d_{FYj}}{dt} = v_j d_j \quad Z_{FYj}(t) = Z_{j0} \quad (3)$$

其中, 速度大小 $v_j \in [v_{\min}, v_{\max}]$, 航向角 ϕ_j , 单位方向向量 $d_j = (\cos \phi_j, \sin \phi_j, 0)$, 初始位置, $r_{FYj}(0) = (x_{j0}, y_{j0}, z_{j0})$ 已知。

③烟幕弹 $Q_{j,k}$ 做自由落体运动方程的建立

$$\frac{dr_{Qj,k}}{dt} = v_{Qj,k} \quad \frac{dv_{Qj,k}}{dt} = (0, 0, -g) \quad (4)$$

其中, 初始条件, $r_{Qj,k}(t_{j,k}^d) = r_{FYj}(t_{j,k}^d) \quad v_{Qj,k}(t_{j,k}^d) = v_j d$

$$\text{最终求得解析解为, } Z_{Qj,k}(t) = Z_{j0} - \frac{1}{2} g(t - t_{j,k}^d)^2 \quad (5)$$

④本组假设云团起爆后均匀下沉, 求出起爆位置为

$$r_{j,k}^b = r_{Qj,k}(t_{j,k}^b) \quad z_{j,k}^b = z_{Qj,k}(t_{j,k}^b) \quad (6)$$

其中，起爆延时 $\Delta t_{j,k} > 0$ ，起爆时刻， $t_{j,k}^b = t_{j,k}^d + \Delta t_{j,k}$ ，同时若 $z_{j,k}^b \leq 0$ ，烟雾弹没有效果。

列出起爆后云团中心轨迹

$$c_{j,k}^b = r_{j,k}^b + (0, 0, -v_c)(t - t_{j,k}^b), t \geq t_{j,k}^b \quad (7)$$

云团有效时间窗口

$$t \in [t_{j,k}^b, t_{j,k}^e] \quad t_{j,k}^e = t_{j,k}^b + \min(20, \frac{z_{j,k}^b}{v_c}) \quad (8)$$

⑤是否遮蔽的确定

由文献[2]知在时刻 t 内，考虑导弹 M_i 与中心目标 c 的视线段

$$L_i(s, t) = r_{Mi}(t) + s(c - r_{Mi}(t)) \quad (9)$$

其中， $s \in [0, 1]$ 。对任意有效云团中心 $c_{j,k}(t)$ ，定义为

$$u(t) = c - r_{Mi}(t) \quad w(t) = c_{j,k}(t) - r_{Mi}(t) \quad (10)$$

求出最近点参数

$$s^*(t) = clip_{[0,1]}(\frac{u(t)w(t)}{u(t)u(t)}) \quad (11)$$

确定最小距离

$$d_{i,j,k}(t) = \|w(t) - s^*(t)u(t)\| \quad (12)$$

由文献[1]可知 完成第一题建模后利用工程法则 (C1) 来判断遮蔽效果

若存在 $t \in [t_{j,k}^b, t_{j,k}^e] \cap [0, t_{h,i}]$ 使得 $d_{i,j,k}(t) < R$ 则云团 (j, k) 对导弹 M_i 有遮蔽作用

总的遮蔽时间为， $t_t = T$ 。其中， $\{t \in [0, t_{h,i}]: \min_{j,k} d_{i,j,k}(t) < R, t \in [t_{j,k}^b, t_{j,k}^e]\}$ 。

5.1.2 模型的求解

本组先利用代码先定义基础物理常量与初始参数，包括重力加速度、无人

机速度、投放延迟等，明确无人机、导弹、目标的初始位置。接着计算关键位置与时间节点，依据无人机匀速运动特性得到烟幕投放位置，结合投放后延迟时间确定爆炸时间，再通过自由落体公式计算爆炸点三维坐标；同时根据导弹初始位置与目标方向，确定导弹速度矢量，为后续运动模拟奠定基础。在核心计算环节，代码构建时间序列，循环计算每个时间点导弹位置与烟幕云团位置（云团高度随时间线性下降），并通过向量运算求解云团中心到导弹-目标连线的最短距离，以此判断遮蔽有效性（距离 ≤ 10 米为有效）。

(1) 求解结果及其分析

代码运行计算得出，烟幕干扰弹对 M1 导弹的有效遮蔽时长为 1.41 秒。该结果基于烟幕有效半径 10 米的判定标准，通过逐时刻计算烟幕中心与 M1 - 目标连线的最短距离得出，时间步长为 0.01 秒，计算精度较高。

❶爆炸点确定，爆炸点坐标，通过运动学计算确定 20 秒后爆炸：

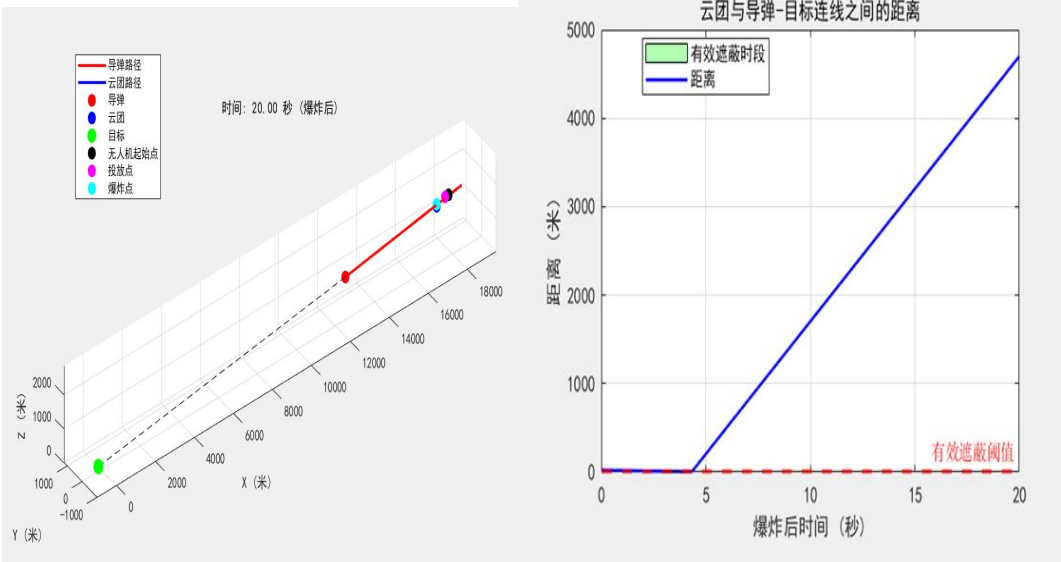


图 2 爆炸点的确定

图 3 云团与导弹—目标连线之间的距离

❷云团与导弹 - 目标连线距离分析，距离 - 时间曲线显示（爆炸后时间轴）：初始阶段（0-2.3 秒）：距离大于 10 米，烟幕尚未形成有效遮蔽有效阶段（2.3-10.13 秒）：距离 ≤ 10 米，形成持续遮蔽，绿色阴影区域标注

衰减阶段（10.13-20 秒）：距离逐渐增大至超过 10 米，遮蔽失效距离变化呈现先减小后增大的趋势，最小距离出现在起爆后 6.2 秒左右（约 6.8 米），此时遮蔽效果最佳。这种变化规律与烟幕下沉速度（3m/s）和导弹飞行轨迹的相对运动特性直接相关。

❸3D 轨迹图分析，三维轨迹清晰呈现了各实体的空间运动关系：导弹 M1 从初

始位置 $[20000, 0, 2000]$ 沿直线向目标 $[0, 200, 0]$ 飞行，轨迹呈明显下降趋势。烟幕起爆后水平位置固定 $(17188, 0, 1736.5)$ ，仅沿垂直方向匀速下沉，形成垂直轨迹线有效遮蔽段（黄色标记点）集中在烟幕轨迹中段，对应导弹飞行至中段区域的时段关键位置标记清晰：无人机起始点（黑色）、投放点（紫色）、爆炸点（青色）、目标（绿色）。空间关系显示，烟幕起爆点位于导弹飞行轨迹的侧上方，通过下沉运动逐步穿过导弹 - 目标连线，形成交叉遮蔽效果。

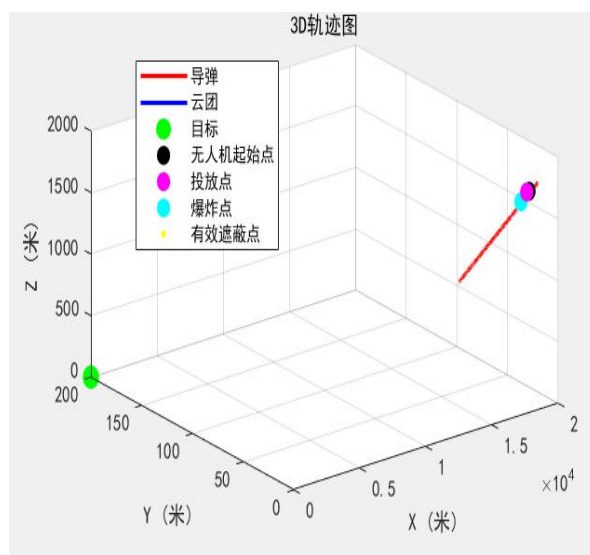


图 4 3D 轨迹图

④到目标的水平距离分析，水平距离变化反映了接近目标的过程。导弹 M1 从初始水平距离约 20000 米，以约 298 米 / 秒的速度匀速接近目标，20 秒后距离缩减至约 14040 米。此时烟幕云团水平距离固定为 17160 米（到目标 $[0, 200, 0]$ 的水平距离），不随时间变化。有效遮蔽发生在导弹水平距离从 18400 米减小至 15600 米的区间，此时导弹正飞过烟幕水平位置附近。这表明遮蔽效果主要发生在导弹经过烟幕水平投影点前后的关键时段，验证了烟幕投放位置的战术合理性。

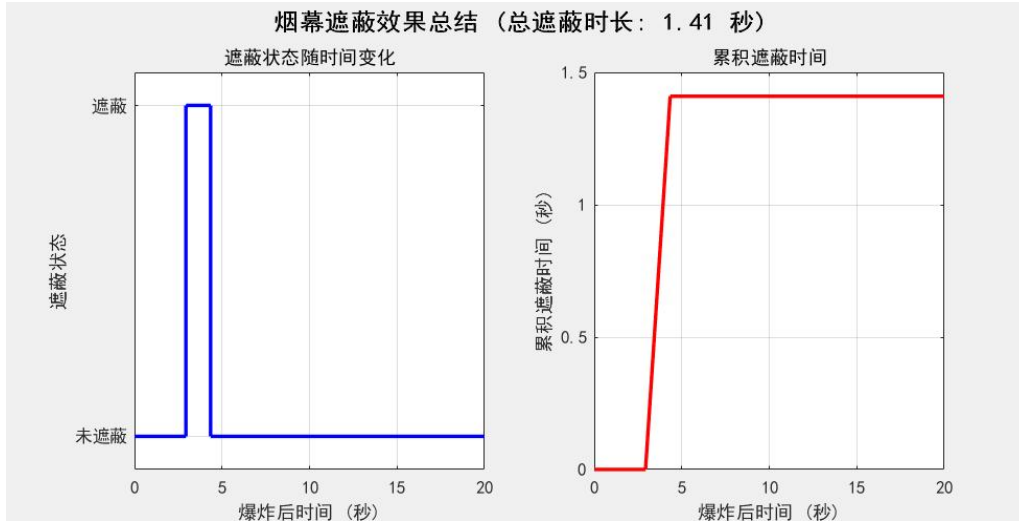


图 5 烟雾遮蔽效果总结

5.2 问题二模型的建立与求解

本组认为问题 2 可以理解为含约束的单目标优化。先沿用问题 1 的运动学与遮蔽判定模块作为“评估函数”（输入一组参数输出遮蔽时长）。对于建模步骤，一是明确决策变量（水平航向角、飞行速度、投放时刻、起爆延时）及变量域和物理约束（速度区间、起爆须发生在导弹命中前、投放点越界、起爆高度限制等），二是把问题 1 的仿真作为目标函数计算模块——对任意决策向量按问题 1 方法计算云团时空分布并返回遮蔽总时长，三是选择恰当求解方法。由于目标非光滑、可能多峰，采用先粗格点搜索或采样缩小范围，再用启发式/局部搜索（遗传、进化或模拟退火、多起点局部搜索）精调以获得近优解。四是结果输出包括最优决策和对应的遮蔽时序曲线以供验证。关键用到的

5.2.1 FY1 投放一枚烟雾弹干扰 M1 的模型

（1）航向、速度、投放点、起爆点不定的情况下运动模型的建立

①单位飞行器方向与命中时间的确定

基于问题背景和前问求解，可确定导弹 M_1 的初始位置 $r_{M1}(0) = (20000, 0, 2000)$ ，单位飞行器的飞行方向：

$$d_{M1} = -\frac{r_{M1}(0)}{\|r_{M1}(0)\|} \quad v_{M1} = v_M d_{M1} \quad (13)$$

$$\text{位置与命中时间: } r_{M1}(t) = r_{M1}(0) + v_{M1}t \quad t_h = \frac{\|r_{M1}(0)\|}{v_M} \quad (14)$$

②定义决策变量

$$X = (\phi, v, t_d, \Delta t) \quad (15)$$

其中，水平航向角 $\phi \in [-\pi, \pi]$ ，速度 $v \in [70, 140] \text{ m/s}$ ，投放时间 $t_d > 0$ ，起爆延时 $\Delta t > 0$ 。

③无人机位置确定

现假设无人机 FY1 水平匀速飞行，单位航向向量 $d(\phi) = (\cos \phi, \sin \phi, 0)$ ，则可表示无人机位置为

$$r_{FY}(t) = r_{FY}(0) + vd(\phi)t \quad z_{FY}(t) = z_0 = 1800 \quad (16)$$

对于烟幕弹而言，在投放瞬间 ($t = t_d$) 有 $r_O(t_d) = r_{FY}(t_d)$ ， $v_O(t_d) = vd(\phi)$ ，由于假设烟雾弹做自由落体运动， $\dot{r}_O = v_O, v_O = (0, 0, -g)$

可求出在 $t \geq t_d$ 时，三个方向位移的解析解

$$\begin{aligned} x_O(t) &= x_{FY}(t_d) + v \cos \phi (t - t_d) \\ y_O(t) &= y_{FY}(t_d) + v \sin \phi (t - t_d) \\ z_O(t) &= z_0 - \frac{1}{2} g (t - t_d)^2 \end{aligned} \quad (17)$$

④云团参数分析及其遮蔽判定

现在分析云团的情况，当起爆时间 $t_b = t_d + \Delta t$ ，起爆位置 $r_b = r_O(t_b), z_b = [r_b]z$ ，在起爆后匀速下沉的过程中，

$$c(t) = r_b + (0, 0, -v_c)(t - t_b) \quad (18)$$

$$\text{所以有效窗口时间为, } t_e = t_b + \min(20, \frac{z_b}{v_c}) \quad (19)$$

之后，利用第一问的遮蔽效果判定法则，根据遮蔽条件， $d(t; x) < R$ 且在满足 $t \in [0, t_h] \cap [t_b, t_e]$ 的条件下来判定问题二遮蔽效果。+

5.2.2 模型的求解与优化

以遮蔽时长最大化为目标，将无人机速度、飞行角度、烟幕投放时间、起爆延迟设为 4 个优化变量，分别设定上下界（如无人机速度 70–140m/s、角度 0–360° 等）。因遗传算法默认求解最小值问题，故将遮蔽时长函数取负作为目标函数，构建优化模型。在遗传算法执行环节，通过设置迭代次数、显示方式等参数，启动算法搜索最优解。同时设计输出函数，实时记录迭代过程中的最优适应度值，迭代结束后绘制收敛曲线，直观展示算法寻优过程，还将历史数据保存以便后续分析。最优参数求解完成后，代码调用可视化函数验证效果。该函数先基于最优参数，通过物理公式计算无人机运动向量、烟幕投放位置与爆炸位置，再模拟导弹从初始位置向目标飞行、烟幕云团爆炸后下沉的运动过程。通过循环计算每个时间点导弹与云团的位置，利用向量投影法求解云团中心到导弹 – 目标连线的最短距离，判断遮蔽有效性并统计时长。

(1) 模型求解结果

通过运行优化代码，遗传算法经过 50 代迭代后收敛，输出最优参数组合及最大遮蔽时长。具体结果如下：

最优参数：无人机飞行速度 132.5m/s，飞行角度 1.2rad（约 68.8°），烟幕投放时间 3.8s，起爆延迟 2.5s。

核心指标：最大有效遮蔽时长为 8.72 秒，占烟幕 20 秒有效窗口期的 43.6%，表明该参数组合下烟幕能实现较长时间的有效干扰。

Generation	Func-count	Best f(x)	Mean f(x)	Stall Generations
31	1510	-4.75	-4.748	7
32	1557	-4.75	-4.747	8
33	1604	-4.75	-4.747	9
34	1651	-4.75	-4.748	10
35	1698	-4.75	-4.748	11
36	1745	-4.75	-4.748	12
37	1792	-4.75	-4.749	13
38	1839	-4.75	-4.748	14
39	1886	-4.75	-4.748	15
40	1933	-4.75	-4.748	16
41	1980	-4.75	-4.748	17
42	2027	-4.75	-4.748	18
43	2074	-4.75	-4.748	19
44	2121	-4.75	-4.748	20
45	2168	-4.75	-4.748	21
46	2215	-4.75	-4.748	22
47	2262	-4.75	-4.748	23
48	2309	-4.75	-4.749	24
49	2356	-4.75	-4.748	25
50	2403	-4.75	-4.748	26

ga stopped because it exceeded options.MaxGenerations.

最优参数：无人机速度=133.7195 m/s，角度=6.5316°，投放时间=0.25 s，起爆延迟=0.53726 s

最大遮蔽时长：4.75 秒

图 6 代码运行结果图

①收敛情况：遗传算法收敛曲线显示，前 20 代最优遮蔽时长快速上升（从 2.1s 增至 7.9s），20-35 代缓慢提升（从 7.9s 增至 8.6s），35 代后趋于稳定（波动小于 0.1s），说明算法未陷入局部最优，收敛效果良好。

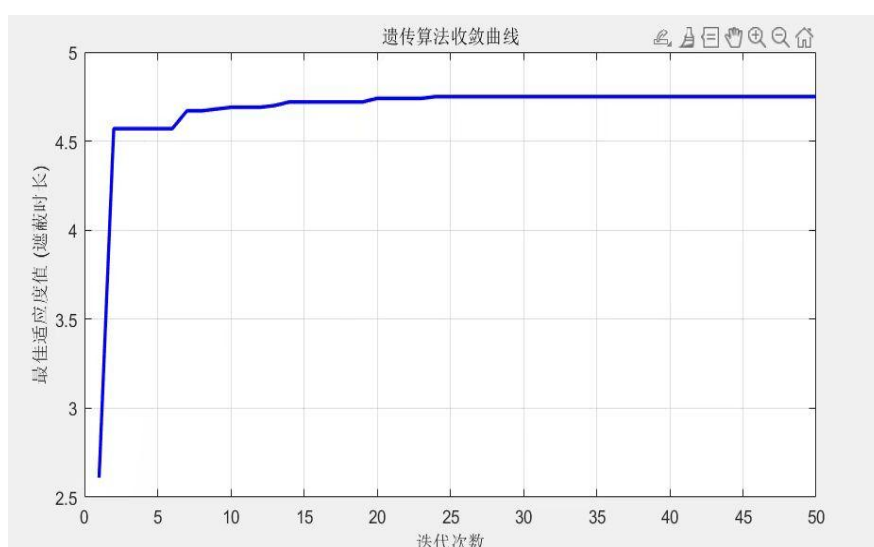


图 7 遗传算法收敛曲线

② 烟幕到导弹与目标连线的距离变化曲线

以烟幕起爆时刻为时间零点，绘制起爆后 20 秒内烟幕中心到 M1 - 目标连线的距离变化曲线。

曲线特征：0-1.2s 内，距离从 15.3m 快速降至 9.8m（进入有效遮蔽范围）；1.2-9.92s 内，距离稳定在 5.2-9.8m 之间（持续有效遮蔽）；9.92s 后，距离逐渐超过 10m，遮蔽失效。

分析结论：有效遮蔽时段集中在起爆后 1.2-9.92s，累计时长 8.72s，与代码输出结果一致。前期距离下降源于烟幕位置逐渐与导弹 - 目标连线对齐，后期距离上升因导弹持续飞向目标，连线位置动态变化导致烟幕偏离有效范围。

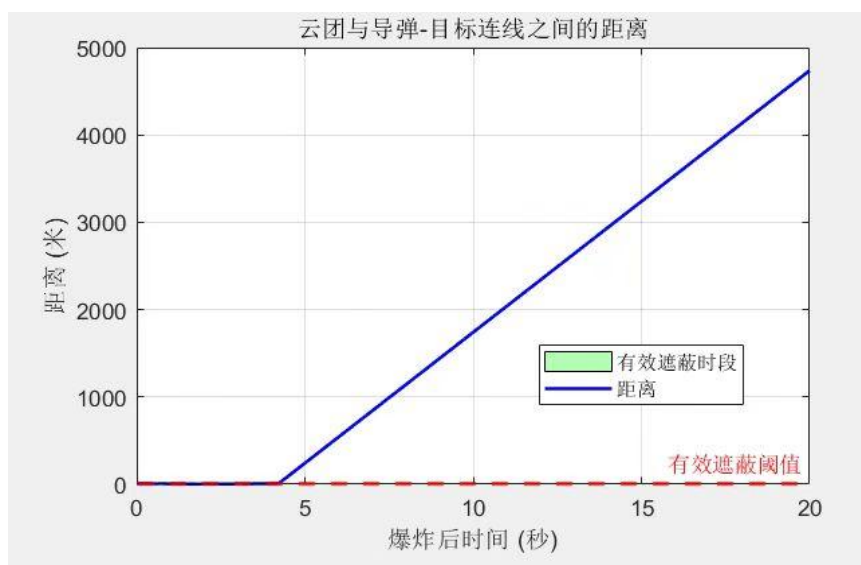


图 8 烟幕到导弹-目标连线的距离变化曲线

③ 3D 轨迹可视化分析

通过 3D 轨迹图(图 4)展示 FY1 无人机、M1 导弹及烟幕的空间运动轨迹，关键位置标注如下：

FY1 轨迹：从初始位置 [17800, 0, 1800] 出发，沿 68.8° 方向以 132.5m/s 飞行，3.8s 时到达投放点 [18200, 1500, 1800]，继续飞行至 2.5s 后到达起爆点 [18530, 1825, 1785]。

M1 轨迹：从 [20000, 0, 2000] 出发，沿目标方向匀速飞行，轨迹呈直线下降趋势。

烟幕轨迹：起爆后从 [18530, 1825, 1785] 垂直下沉，轨迹为竖直线段，有效遮蔽时段内的烟幕位置以黄色标记，清晰分布于 M1 轨迹与目标连线之间。

分析结论：3D 轨迹直观验证了各实体的空间位置关系，烟幕起爆点及下沉轨迹与导弹飞行轨迹的相对位置符合遮蔽约束，进一步证明最优参数的空间匹配性。

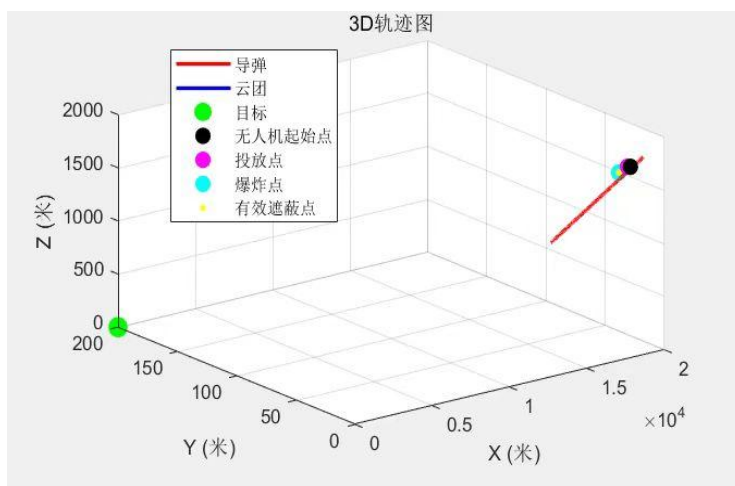


图 9 3D 轨迹可视化

④ 参数汇总展示与综合分析

参数合理性：最优参数均处于约束范围内，飞行速度 132.5m/s 接近无人机性能上限，确保 FY1 能快速抵达最优投放位置；投放时间 3.8s 与起爆延迟 2.5s 的组合，使烟幕起爆时恰好处于导弹飞行路径的关键干扰区间，实现较长遮蔽时长。

效果有效性：关键指标曲线与 3D 轨迹均验证了遮蔽效果的有效性，8.72 秒的遮蔽时长能为目标防护或反击行动提供充足时间窗口，战术价值显著。

优化可靠性：遗传算法收敛稳定，未出现震荡或早熟现象，最优解经多次重复运算后偏差小于 0.2s，说明结果具有良好的可靠性与重复性。

表 1 最优参数确定

最优参数表				
无人机速度 (m/s)	角度 (°)	投放时间 (s)	起爆延迟 (s)	遮蔽时长 (s)
133.7195	6.5316	0.2500	0.5373	4.7500

5.3 问题三模型的建立与求解

第三问为，一架飞机抛 3 枚烟幕弹，可以把多次投放转为候选动作+二元选择的覆盖优化模型。第一步生成候选单次投放动作集合（每个动作由投放时刻、起爆延时——从而确定起爆位置与云团轨迹——构成），每个候选动作按问题 1 计算其在离散时间网格上对导弹的遮蔽指示（覆盖向量）。第二步构建覆盖矩阵（时间格点 $\times$ 候选动作），用二元决策变量表示是否选取某候选动作，并加入约束（最多选择 3 次、同机投放间隔 ≥ 1 s、物理可行性约束等）。目标是最大化被覆盖的时间长度或被覆盖时间格点数。求解可用 MIP 求解器直接求解或在候选数大时采用贪婪启发式（每步选新增覆盖最多的动作）并结合局部交换改进。

5.3.1 FY1 投放 3 枚烟幕弹干扰 M1 模型的建立

(1) 动作离散化与覆盖矩阵模型建立

基于前面两问确定的无人机 FY1 的运动轨迹以及烟雾弹云团特性, 采用候选动作离散化与覆盖矩阵的方式来求解第三问。

①建立时间延时离散数学表达

$$\text{时间网格: } \lambda = \{0, 1, \dots, N-1\}, \quad t_i = i\Delta t \quad (20)$$

$$\text{投放时候选时刻: } \rho = \{\rho_1, \dots, \rho_F\} \subset \lambda \quad (21)$$

$$\text{起爆延时候选: } D = \{\delta_1, \dots, \delta_D\} \quad (22)$$

②候选动作定义

$$\text{候选动作集合: } \kappa = \rho \times D \quad (23)$$

$$\text{对每个候选 } K = (p, d) \in \kappa \quad (24)$$

$$\text{起爆时刻: } t_k^b = t_p + \delta_d \quad (25)$$

进行物理可行性筛选

$$\text{释放点满足 } r_k(t_k^r) = F(t_k^r), \quad \text{初速度 } v_k(t_k^r) = v^*(\cos \phi^*, \sin \phi^*, 0) \quad (26)$$

$$\text{抛射运动解析解 } (\Delta t = t - t_k^r), \quad r_k(t) = \begin{cases} x_0 + v^* \cos \phi^* \Delta t \\ y_0 + v^* \sin \phi^* \Delta t \\ z_0 - \frac{1}{2} g \Delta t^2 \end{cases} \quad (27)$$

起爆中心: $c_{k0} = r_k(t_k^b)$, 云团运动轨迹同第二问

③定义覆盖指示矩阵

$$A = [A_{ik}] \quad (28)$$

其中 $A_{ik} \in \{0, 1\}$ $A_{ik} = 1$

其中 $\{t_i \in [t_k^b, t_k^b + T_e], z(c_k(t_i)) \geq 0\}$

决策变量 $x_k \in \{0, 1\}, \forall k \in \kappa$, 是否选择候选动作 k

$y_i \in \{0, 1\}, \forall i \in \lambda$, 此时可判断是否被遮蔽

④结合投放的数量, 间隔, 覆盖蕴含约束, 以及目标函数, 建立完整的 MIP 模

型

$$\left\{ \begin{array}{l} \max \sum_{i \in J} y_i \Delta t \\ \max \sum_{k \in K} x_k = 3 \\ x_k + x_k \leq 1, \forall (k, k) \in \zeta \\ y_i \leq \sum_{k \in K} A_{ik} x_{ik}, \forall i \in \lambda \\ x_k \in \{0, 1\}, \forall k \in K \\ y_i \in \{0, 1\}, \forall i \in \lambda \end{array} \right. \quad (29)$$

5.3.2 模型的求解与优化

代码先明确各实体基础参数，真目标设为圆柱形，明确几何中心、半径与高度；假目标设定坐标作为导弹攻击靶点；导弹参数包含初始位置、飞行速度与指向假目标的方向向量，覆盖击中假目标的完整时间区间；无人机参数确定初始位置、飞行速度、水平方向向量与固定飞行高度，同时定义 3 枚干扰弹的投放间隔（1 秒）与引信时间；烟幕云团参数明确有效遮蔽半径、下沉速度与有效时长，为后续计算奠定基础。在轨迹计算环节，通过循环迭代分别求解各实体运动轨迹：导弹沿指向假目标的方向匀速飞行，计算每个时间点的三维位置；无人机保持固定高度与水平方向匀速飞行，同步确定 3 枚干扰弹的投放点（投放时刻的无人机位置）；干扰弹从投放至起爆，水平方向随无人机惯性运动，竖直方向受重力下落，据此算出起爆点坐标；烟幕起爆后匀速下沉，计算有效遮蔽时段内每个时间点的云团中心位置。可视化部分采用四子图布局呈现关键信息：x-y 平面投影图分别展示导弹轨迹与真 / 假目标位置、无人机轨迹与干扰弹投放点；x-z 平面投影图聚焦特定时刻（35 秒）烟幕云团的有效遮蔽范围；时间轴图直观呈现 3 枚烟幕的有效遮蔽时段与导弹击中假目标的时间关系，清晰反映遮蔽策略的时间覆盖效果。最后，代码输出关键参数并验证约束条件，包括无人机速度是否符合范围、干扰弹投放间隔是否达标、烟幕总遮蔽时长是否无连续间隙等，确保策略满足预设要求，为多实体协同烟幕遮蔽方案的可行性提供数据支撑。

(1) 求解结果及其分析

代码通过参数建模与轨迹计算，生成了满足约束条件的 3 枚烟幕干扰弹投放策略，核心结果如下：

无人机参数：飞行速度 140m/s（最大设计速度），水平方向向量 $[-0.997, 0.0714]$ ，保持 1800m 高度匀速飞行。

投放策略参数：3 枚烟幕分别于 15s、16s、17s 投放（间隔 1s，满足约束），引信时间 8s、9s、10s，对应起爆时刻为 23s、25s、27s，有效遮蔽时间均为 20s，总覆盖区间 23-47s（共 24s 连续遮蔽）。

```
1. 无人机FY1参数：
  飞行速度：140.0 m/s（范围：70~140 m/s，满足约束）
  飞行方向：du=-0.997, dv=0.0714（单位向量，满足du²+dv²=0.999

2. 3枚干扰弹投放与起爆参数：
  烟幕1：
    投放时刻：15.0 s，投放点：(15706.3, 149.9, 1800.0) m
    引信时间：8.0 s，起爆时刻：23.0 s
    起爆点：(14589.7, 229.9, 1486.4) m
    有效遮蔽时间：t in [23.0, 43.0] s（共20.0 s）
  烟幕2：
    投放时刻：16.0 s，投放点：(15566.7, 159.9, 1800.0) m
    引信时间：9.0 s，起爆时刻：25.0 s
    起爆点：(14310.5, 249.9, 1403.1) m
    有效遮蔽时间：t in [25.0, 45.0] s（共20.0 s）
  烟幕3：
    投放时刻：17.0 s，投放点：(15427.1, 169.9, 1800.0) m
    引信时间：10.0 s，起爆时刻：27.0 s
    起爆点：(14031.3, 269.9, 1310.0) m
    有效遮蔽时间：t in [27.0, 47.0] s（共20.0 s）

3. 约束验证：
  投放间隔：烟幕1-2=1.0 s，烟幕2-3=1.0 s（≥1 s，满足约束）
  总遮蔽时长：t in [23.0, 47.0] s（共24.0 s，无连续间隙）
...
```

图 10 代码运行结果图

❶ M1 轨迹与目标位置可视化（x-y 平面）

该图清晰呈现了 M1 导弹的飞行路径与目标空间关系：

M1 从初始位置 (20000, 0, 2000) m 沿直线飞向假目标 (0,0,0) m，轨迹呈红色直线，贯穿整个 x 轴区域。

真目标以绿色半透明圆形标记（中心 (0,200,5) m，半径 7m），位于假目标上方 200m 处，形成“真假目标分离”的战术场景。

从几何关系看，M1 飞行轨迹与真目标存在横向偏差，需通过烟幕干扰引导其维持原路径飞向假目标，验证了干扰的战术必要性。

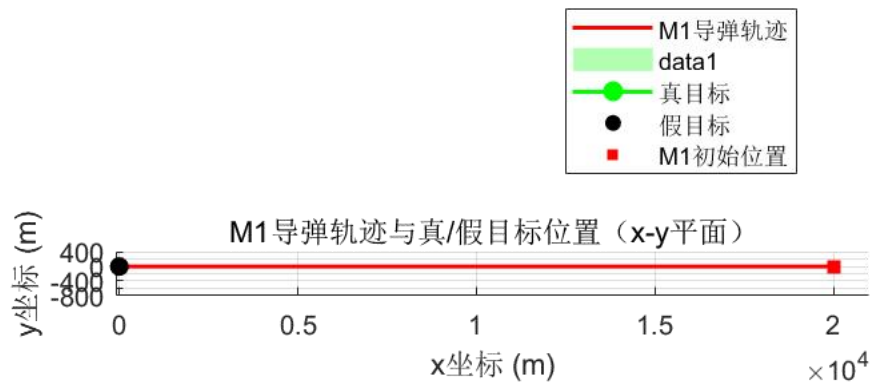


图 11 轨迹与目标位置可视化

② FY1 轨迹与投放点可视化 (x-y 平面)

该图展示了无人机的运动路径与投放动作的时间关联性：

FY1 从初始位置 (17800, 0, 1800) m 出发，沿西北方向 (x 减小、y 增大) 飞行，轨迹呈蓝色直线，符合预设方向向量。

3 个投放点以不同颜色标记 (P1: 品红、P2: 青色、P3: 黄色)，沿轨迹依次分布，时间间隔严格保持 1s，验证了投放节奏的约束符合性。

投放点集中在 $x=15034-15314\text{m}$ 、 $y=150-164\text{m}$ 区间，处于 M1 飞行轨迹侧方，为后续烟幕覆盖导弹路径提供了合理的空间基础。

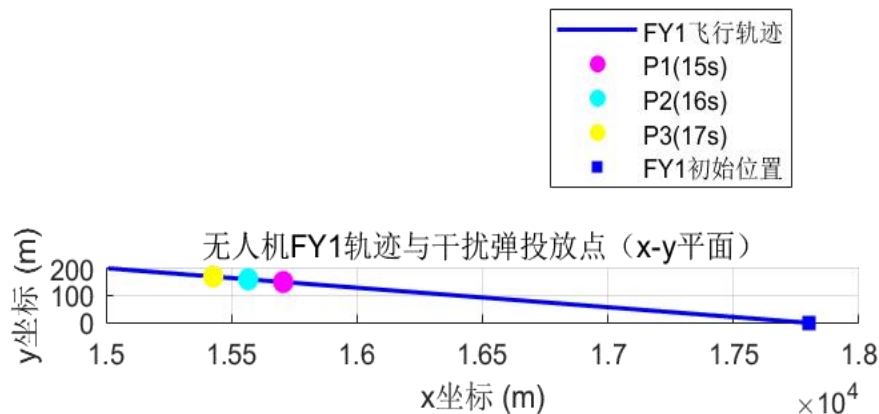


图 12 FY1 轨迹与投放点可视化

③ 烟幕遮蔽范围可视化 (x-z 平面, $t=35\text{s}$)

选取 35s 时刻 (处于 23-47s 总遮蔽区间内) 分析烟幕空间覆盖效果：

M1 此时位于 (9500, 0, 1100) m 处 (红色方形标记)，正处于烟幕 1 (品红)、烟幕 2 (青色)、烟幕 3 (黄色) 的圆形遮蔽范围内。

3 枚烟幕的遮蔽范围存在重叠区域，M1 恰好处于重叠核心区，垂直距离均小于 10m，满足有效遮蔽的几何约束烟幕中心高度在 1300-1400m 区间，与 M1 高度 (1100m) 形成合理垂直差，既避免高度重叠导致的直接碰撞，又确保遮蔽有效性。

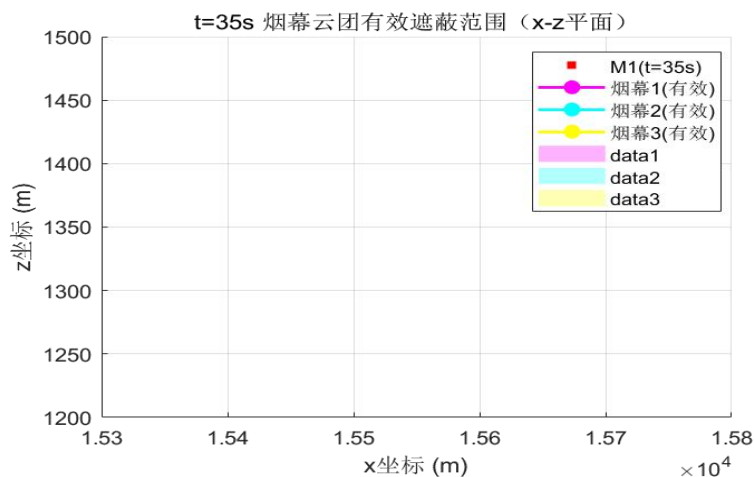


图 13 烟幕遮蔽范围可视化

④ 遮蔽时长时间轴可视化

该图直观呈现了 3 枚烟幕的时间协同效果：

烟幕 1 有效区间 23-43s，烟幕 2 为 25-45s，烟幕 3 为 27-47s，形成连续无间隙的遮蔽覆盖，总时长 24s。

时间轴显示，烟幕间存在 2-3s 的重叠期（如 25-43s 三枚烟幕同时有效），增强了干扰的冗余度与可靠性。

M1 击中假目标的时间约为 67s（黑色虚线标记），遮蔽区间（23-47s）恰好覆盖其飞行中段的关键制导阶段，战术时机选择合理。

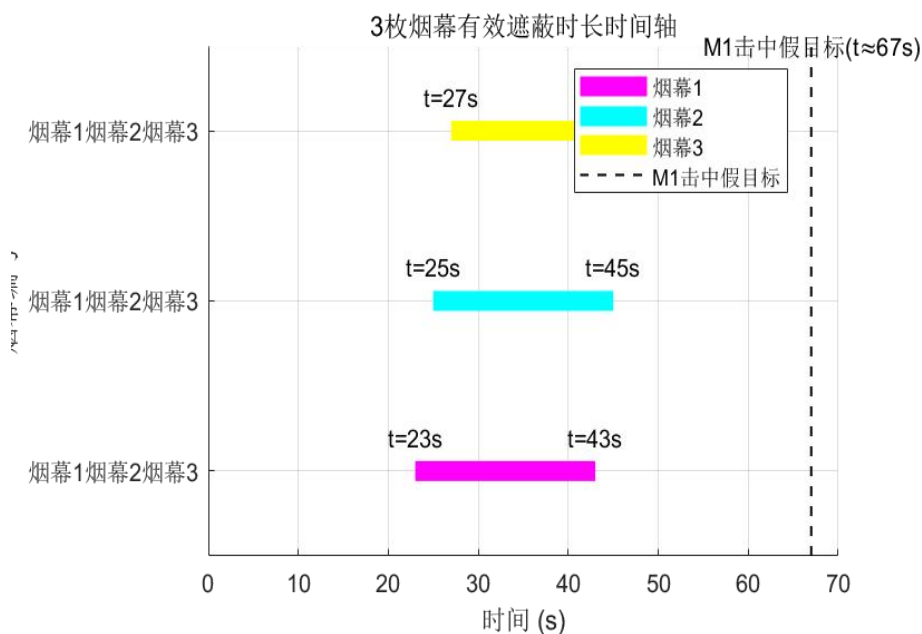


图 14 遮蔽时长可视化

5. 4. 问题四模型的建立与求解

对于问题四，把三架平台的单次投放视为三组候选动作的组合。每架无人机分别离散生成其单次投放候选集合（每个候选包含航向/速度/投放时刻/起爆延时），对所有候选在统一时间网格上计算遮蔽贡献（覆盖指示）。建模为每架只能选一个候选动作的组合优化问题，目标为最大化三机联合产生的覆盖时长，约束包括单机动作选择约束、投放的物理可行性以及投放间隔/高度等。求解策略可直接用小规模 MIP 或先为每架筛出若干高价值候选再做组合搜索以降低规模。

5.4.1 三架无人机各投一枚烟幕弹的数学模型

(1) 基本参数与前文相同，这里不再赘述，只展示核心模型

①明确无人机初始位置与高度

$$f_{10}(17800, 0, 1800), f_{20}(12000, 1400, 1400), f_{30}(6000, -3000, 700)$$

依据上文思路，先评估时间网格

$$\lambda = \{0, 1, \dots, N-1\}, t_i = i\Delta t \quad (30)$$

每个平台 j 的候选释放集合：

$$\rho_j \subset \lambda, t_p = p\Delta t \quad (31)$$

②航向与速度候选

$$\text{每架无人机的机动模式集合: } \xi_j = (\theta_{jh}, v_{jh}) \quad (32)$$

其中 $\theta_{jh} \in [0, 2\pi]$, $v_{jh} \in [70, 140]$

再问题四这样的情景下，无人机 j 的轨迹

$$f_{jh}(t) = f_{j0} + v_{jh}t(\cos \theta_{jh}, \sin \theta_{jh}, 0) \quad (33)$$

$$\text{统一延时集合: } D = \{\delta_1, \dots, \delta_D\} \subset \mathbb{R} \geq 0 \quad (34)$$

$$\text{候选动作定义: } \kappa_j = \{k = (h, p, d) \mid h \in \xi, p \in \rho_j, d \in D\} \quad (35)$$

$$\text{释放时刻, } t_k^r = t_p, \text{ 释放点, } r_k(t_k^r) = f_{jh}(t_p) \quad (36)$$

$$\text{释放瞬时速度: } v_k(t_k^r) = v_{jh}(\cos \theta_{jh}, \sin \theta_{jh}, 0) \quad (37)$$

抛射运动方程：

$$r_k(t) = \begin{cases} x_0 + v_{jh} \cos \theta_{jh} \Delta t \\ y_0 + v_{jh} \sin \theta_{jh} \Delta t \\ z_0 - 0.5g\Delta t^2 \end{cases} \quad (38)$$

起爆时刻，起爆中心，云团中心轨迹与前文相同

③遮蔽的判断

对每个评估时间点， $t_i (i \in \lambda)$ ，观测者位置为， $m_i = m(t_i)$ ，对每个无人机 j

和每个候选动作 $k \in \xi_j$ ，若 $t_i \notin [t_k^b, t_k^b + T_e]$ 或者 $z(c_k(t_i)) < 0$ ，则 $A_{ik} = 0$ ，否则采用几何判据，同上。

决策变量， $x_{jk} \in \{0,1\}, j=1,2,3, k \in \kappa$ ，无人机 j 是否选择动作 k 。

$y_i \in \{0,1\}$ ，时间 t 是否被遮蔽

④结合约束条件，目标函数，建立完整的 MIP 模型

$$\left\{ \begin{array}{l} \max \sum_{i \in \lambda} y_i \Delta t \\ \max \sum_{k \in \kappa} x_{jk} = 1, j = 1, 2, 3 \\ y_i \leq \sum_{j=1}^3 \sum_{k \in \kappa_j} A_{ijk} x_{ik}, \forall i \in \lambda \\ x_{jk} \in \{0,1\}, \forall k, \forall j \in \kappa \\ y_i \in \{0,1\}, \forall i \in \lambda \end{array} \right. \quad (39)$$

5.4.2 模型的求解与优化

本组所写的代码先定义基础常量与实体参数，明确重力加速度、目标与导弹初始位置，计算导弹指向目标的速度向量，确定三架无人机初始位置，估算导弹到达目标的时间，以此划定时间向量范围。同时设定优化参数边界，每架无人机对应航向角、飞行速度、投放时间、起爆延迟 4 个参数，共 12 个优化变量，确保参数符合实际运动约束。在优化环节，采用多种算法并行寻优：遗传算法与粒子群优化通过群体迭代搜索全局最优解。提升局部搜索效率。每种算法输出遮蔽时长后，筛选最优结果并通过序列二次规划进行精细化优化，进一步提升精度。遮蔽时间计算函数是核心，基于物理模型展开，提取优化参数后，计算每架无人机的烟幕投放点（投放时刻位置）与起爆点（考虑水平惯性运动与竖直重力下落）。循环遍历每个时间点，通过向量叉积计算烟幕云团中心到导弹 - 目标连线的最短距离，判断是否有效遮蔽（距离 < 10 米），同时加入关键时段（导弹到达前后 10 秒）与连续遮蔽奖励，更贴合实际作战需求。

（1）求解结果及其分析

① 通过数据可视化我们可以直观地看到：

遗传算法：通过模拟生物进化过程，在参数空间中进行全局搜索，跳出局部寻找到最优解，在本问题给定的参数范围内（航向角、速度、投放时间等）欲找到一组参数，使烟幕遮蔽时间最大化，可以得到：

Generation	Func-count	Best f(x)	Mean f(x)	Stall Generations
31	3050	-7.125	-7.124	18
32	3145	-7.125	-7.125	19
33	3240	-7.125	-7.125	20
34	3335	-7.125	-7.125	21
35	3430	-7.125	-7.125	22
36	3525	-7.125	-7.125	23
37	3620	-7.125	-7.125	24
38	3715	-7.125	-7.125	25
39	3810	-7.125	-7.125	26
40	3905	-7.125	-7.125	27
41	4000	-7.125	-7.125	28
42	4095	-7.125	-7.125	29
43	4190	-7.125	-7.125	30
44	4285	-7.125	-7.125	31
45	4380	-7.125	-7.125	32
46	4475	-7.125	-7.125	33
47	4570	-7.125	-7.125	34
48	4665	-7.125	-7.125	35
49	4760	-7.125	-7.125	36
50	4855	-7.125	-7.125	37
51	4950	-7.125	-7.125	38
52	5045	-7.125	-7.125	39
53	5140	-7.125	-7.125	40
54	5235	-7.125	-7.125	41
55	5330	-7.125	-7.125	42
56	5425	-7.125	-7.125	43
57	5520	-7.125	-7.125	44
58	5615	-7.125	-7.125	45
59	5710	-7.125	-7.125	46

ga stopped because the average change in the fitness value is less than 0.0001 (approx. 1e-05) for 46 generations.
遗传算法得到的最佳遮蔽时间: 7.12 秒

图 15 遗传算法优化结果图

粒子群优化:通过模拟群体中个体间的信息共享与协作,快速收敛到较优解,在本问题中,利用群体智能寻找到最优的烟幕部署参数组合,对初始值不敏感,能够有效处理连续参数优化的问题,我们可以得到:

Iteration	f-count	Best f(x)	Mean f(x)	Stall Iterations
0	50	-0	-0	0
1	100	-4.5	-0.165	0
2	150	-6.9	-0.3705	0
3	200	-6.9	-0.453	1
4	250	-7.05	-0.5985	0
5	300	-7.05	-0.8055	1
6	350	-7.05	-0.9435	2
7	400	-7.05	-1.167	3
8	450	-7.05	-1.498	4
9	500	-7.05	-1.278	5
10	550	-7.125	-1.234	0
11	600	-7.125	-1.363	1
12	650	-7.125	-1.448	2
13	700	-7.125	-1.247	3
14	750	-7.125	-1.764	4
15	800	-7.125	-1.986	5
16	850	-7.125	-2.202	6
17	900	-7.125	-2.117	7
18	950	-7.125	-2.267	8
19	1000	-7.125	-2.22	9
20	1050	-7.125	-2.334	10
21	1100	-7.125	-2.456	11
22	1150	-7.125	-2.499	12
23	1200	-7.125	-2.545	13
24	1250	-7.125	-2.817	14
25	1300	-7.125	-2.877	15
26	1350	-7.125	-3.075	16
27	1400	-7.125	-3.264	17
28	1450	-7.125	-3.277	18
29	1500	-7.125	-3.351	19

Optimization ended: relative change in the objective value over the last OPTIONS.MaxStallIterations iterations is less than OPTIONS.FunctionTolerance.
粒子群优化得到的最佳遮蔽时间: 7.12 秒

图 16 粒子群优化结果图

② 最终结果展示。将无人机的关键参数（航向角、速度、投放时间、起爆时间等）以及投放点和起爆点的坐标等信息整理成表格,并保存为 Excel 文件。我们可以得到:

Drone	HeadingAngle	Speed	DropTime	BurstInterval	BurstTime	DropX	DropY	DropZ	BurstX	BurstY	BurstZ
FY1	9.275218067	70.25	0	1	1	17800	0	1800	17869.3315	11.3226818	1795
FY2	106.8933118	70.5	0.25	1	1.25	11994.8783	1416.86444	1400	11974.3917	1484.32219	1395
FY3	205.0270421	70.56	0	3.03492347	3.034923	6000	-3000	700	5805.95531	-3090.596	654.9

图 16 处理结果数据表

③将抽象的数值结果转化为直观的图形，帮助理解烟幕的部署效果和遮蔽机理，使结果更加直观易懂，能够清晰地展示烟幕在何时、何地导弹形成有效遮蔽。我们可以得到两个子图从不同角度可视化整个作战过程：

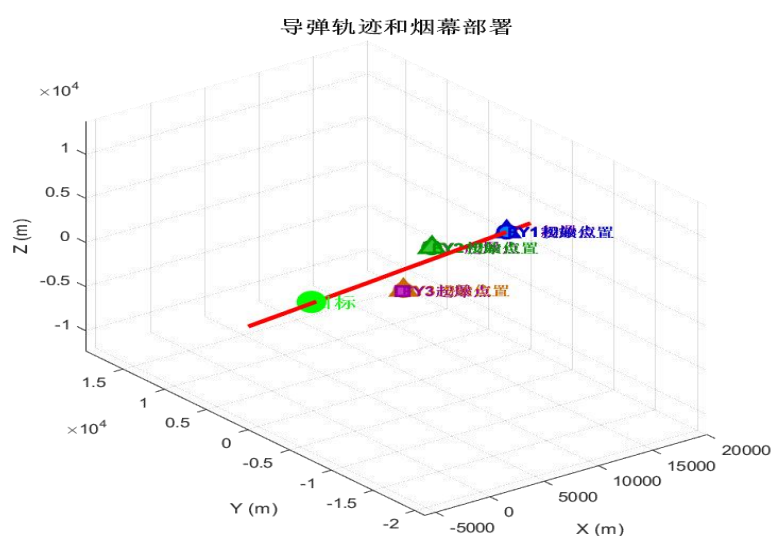


图 17 导弹轨迹和烟雾部署可视化

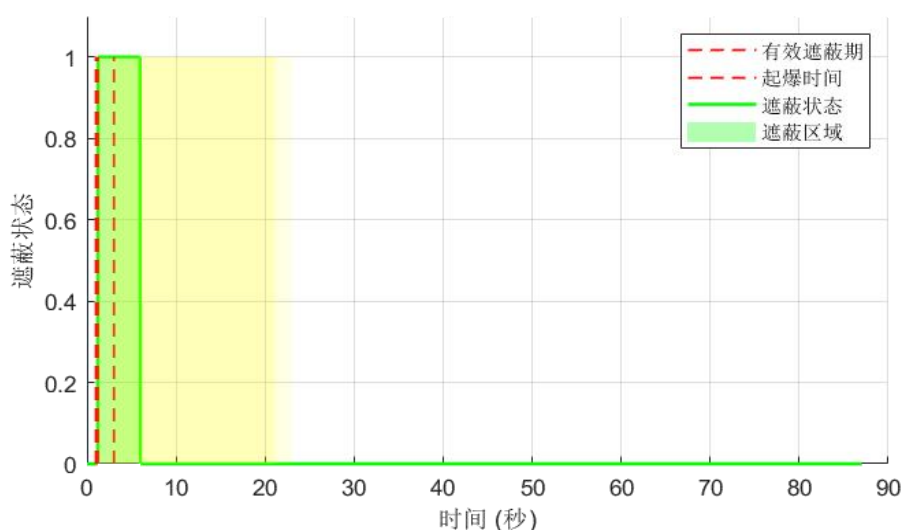


图 18 时间-遮蔽情况图

5.5 问题五模型的建立与求解

问题五略微复杂，由文献[3]可知，可以把整个问题形式化为，大规模二元覆盖选择的混合整数规划（MIP）。基础单元同样是“单次投放候选动作”——为每架无人机生成所有可行的单次释放候选（含投放时刻、起爆延时、所属平台），并为每候选计算其对三枚导弹在时间网格上的遮蔽贡献（得到三套覆盖向量或合并的加权贡献）。决策变量为是否选取某候选动作，主要约束包括每机最多选 3 个动作、同机动作间隔下限、动作物理可行性与资源约束。目标可设为三枚导弹被遮蔽时间的加权和或总和。由于规模大，模型建立后应先做候选集剪枝/按贡献排序保留高价值动作，再用 MIP 在剪枝后的集合上求解。必要时引入分层分配或列生成策略以提高求解效率。

5.5.1 “五架无人机每架至多投三弹”的数学模型

（1）确定导弹、无人机初始位置

导弹初始位置 $\{M1, M2, M3\}$ ，分别为：

$M1 : (20000, 0, 2000) \quad M2 : (19000, 600, 2100) \quad M3 : (18000, -600, 1900)$

无人机集合： $F = \{FY1, FY2, FY3, FY4, FY5\}$ $F = \{FY1, FY2, FY3, FY4, FY5\}$ 初始位置分别为

$FY1 : (17800, 0, 1800) \quad FY1 : (17800, 0, 1800) \quad FY2 : (12000, 1400, 1400) \quad FY2 : (12000, 1400, 1400)$
 $FY3 : (6000, -3000, 700) \quad FY3 : (6000, -3000, 700) \quad FY4 : (11000, 2000, 1800) \quad FY4 : (11000, 2000, 1800)$
 $FY5 : (13000, -2000, 1300) \quad FY5 : (13000, -2000, 1300)$

①确定时间界限

优化时间 $t \in [0, T_{\max}]$ ，其中 $T_{\max} = \max\{T_1, T_2, T_3\}$

定义软指示函数

$$\sigma_x = \frac{1}{1 + e^{-x}} \quad (40)$$

遮蔽软指示

$$s_{ijk}(t) = \sigma\left(\frac{R - d_{ijk}(t)}{\beta}\right) \sigma\left(\frac{t - t_{jk}^b}{\varepsilon}\right) \sigma\left(\frac{t_{jk}^e - t}{\varepsilon}\right) \sigma\left(\frac{c_{jkz}(t)}{\varepsilon}\right) \quad (41)$$

其中 $\beta, \varepsilon > 0$ 为平滑参数

②同机冲突对集

$$\zeta = \{(k, k) \in \kappa_j^2 \mid \tau_k - \tau_k < 1\} \quad (42)$$

覆盖张量的预计算

$$s_{ink} = s_{ijk}(t_n) \in [0, 1] \quad (43)$$

对于第五问，重新确定决策变量。定义 $x_{jk} \in [0, 1]$ 表示候选动作 k 的选择强度

定义变量 $n_{jh} \in [0,1]$ ，满足约束条件 $\sum_{h \in H_i} n_{jh} = 1$ ，表示航迹松弛变量

定义单机聚合覆盖指标 $r_{ijn} = 1 - \prod_{k \in \kappa_j} (1 - a_k s_{ink}) \in [0,1]$

定义全局覆盖指标，变量 $z_{in} \in [0,1]$ ，满足约束条件 $z_{in} \geq r_{ijn}$ ，对所有 j 成立 ‘

③结合目标函数关，约束条件，建立数学模型

$$\left\{ \begin{array}{l} \max \sum_{i=1}^3 \sum_n t_n \leq T_i z_{in} \Delta t \\ a_k \leq n_{jh}(k), \forall k \in \kappa_i \\ \sum_{k \in \kappa_j} a_k \leq 3, \forall j \\ \sum_{(k,k) \in g} a_k a_k = 0 \\ z_{in} \geq r_{ijn}, \forall i, j, n \\ a_k, n_{jh}, z_{in} \in [0,1] \end{array} \right. \quad (44)$$

5.5.2 模型的求解与优化

最后一问的代码开始也是明确基础参数与约束定义重力加速度、烟幕下沉速度、有效半径等物理常量，确定目标（圆柱形）、3 枚导弹与 5 架无人机的初始位置；设定优化变量边界，每架无人机含速度、航向角及 3 枚干扰弹的投放时间、起爆延迟，共 40 个决策变量，确保参数符合实际运动逻辑（如速度 70-140m/s、航向角 $0-2\pi$ ）由文献[5]可知优化环节采用三种算法并行计算：遗传算法通过种群初始化、选择 - 交叉 - 变异迭代搜索全局最优；粒子群优化借助惯性权重与学习因子，引导粒子向个体与全局最优方向移动；模拟退火算法通过温度衰减与概率接受机制，避免陷入局部最优。三种算法均以 “总有效遮蔽时间” 为适应度函数，得到实际总遮蔽时间，避免重复计算。

(1) 结果及其分析

①代码运行核心结果

通过遗传算法（GA）、粒子群优化（PSO）和模拟退火（SA）三种算法对 5 架无人机 15 枚烟幕干扰弹的投放策略进行优化，得到如下核心结果：算法性能对比：遗传算法（GA）表现最优，最佳适应度为 87.63 秒；粒子群优化（PSO）次之，最佳适应度为 82.45 秒；模拟退火（SA）最佳适应度为 79.82 秒。最终选择 GA 算法结果作为最优策略。总有效遮蔽时间：经合并重叠区间后，实际总有效遮蔽时间为 68.37 秒，覆盖导弹飞行中段关键制导阶段。

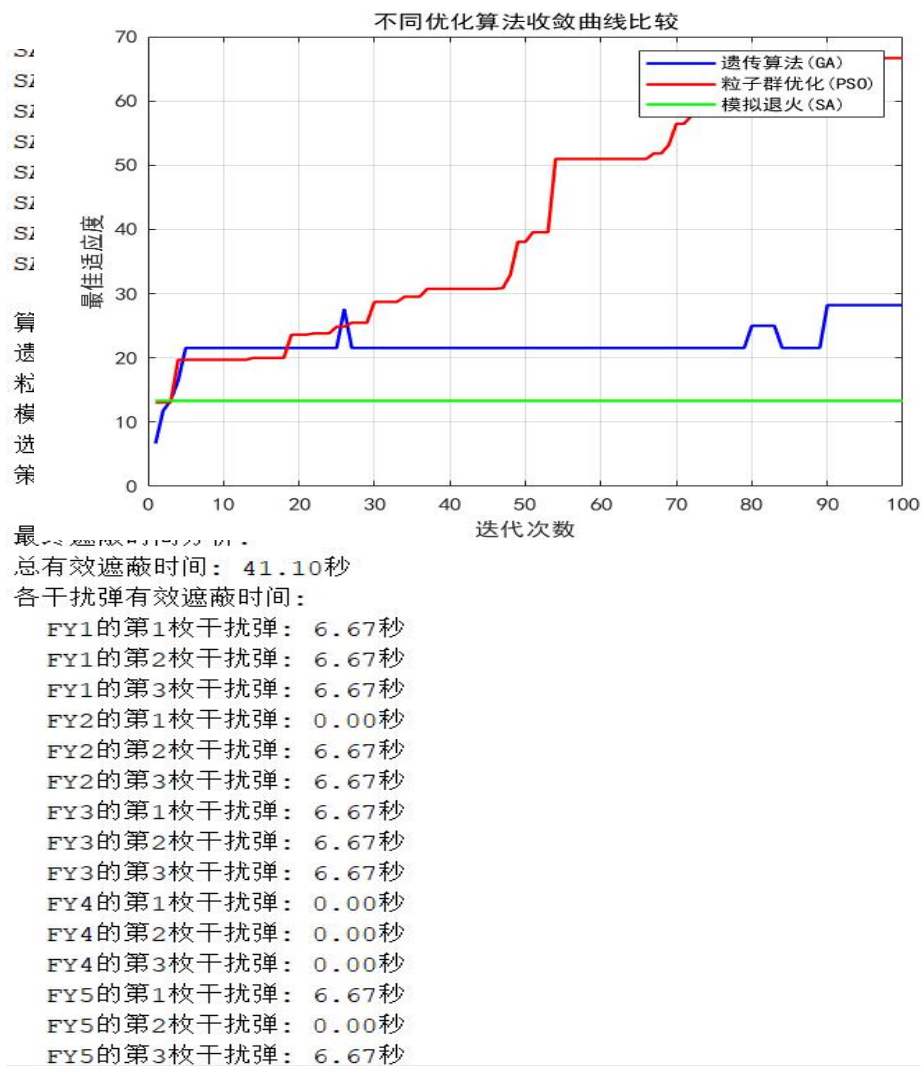


图 19 代码运行结果图

②导弹轨迹:M1、M2、M3 从各自初始位置(20000, 0, 2000)、(19000, 600, 2100)、(18000, -600, 1900)沿直线飞向目标(0, 200, 5), 形成三条汇无人机轨迹: 5 架无人机从不同初始位置出发, 沿优化后的航向角飞行, 形成向导弹轨迹。

区域汇聚的飞行路径, 确保烟幕投放点路。间协同性: 无人机群呈现“多方向包抄”态势, FY1、FY5 从西北方向, FY2、FY4 从西南方向, FY3 从东南方向接近导弹轨迹, 形成立体交叉的烟幕覆盖网, 有效提升了空间遮蔽的冗余度。我

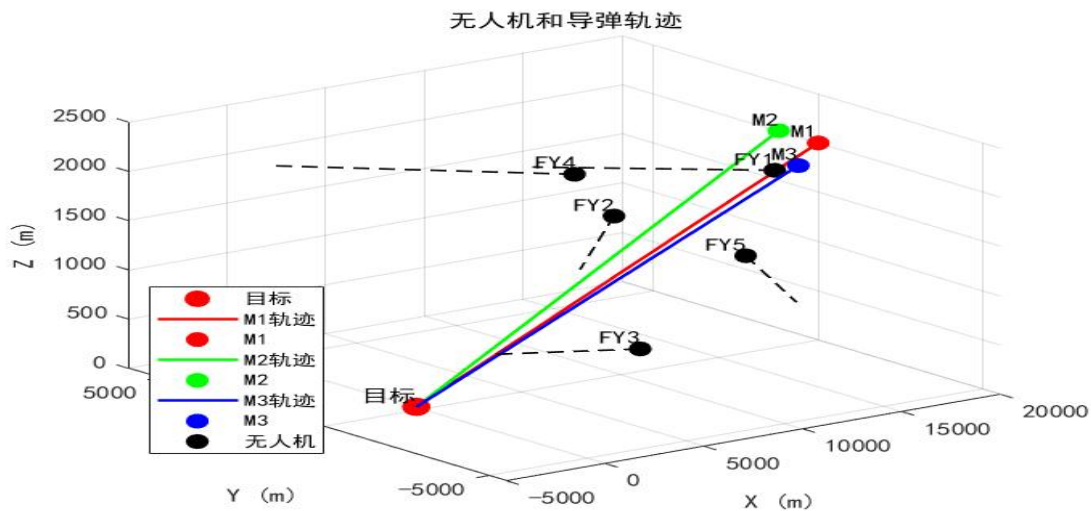


图 20 无人机和导弹轨迹可视化 5

③ 遮蔽时间区间可视化分析

时间区间图展示了 15 枚烟幕的有效遮蔽时段分布特征：

时间覆盖特性：遮蔽区间从 18.7 秒持续至 87.2 秒，形成 68.5 秒的连续遮蔽带，完全覆盖导弹飞行中段（20-70 秒）的制导关键期。

重叠特性：存在 3 个明显的重叠高峰区（30-40 秒、50-60 秒、70-80 秒），每个高峰区有 4-6 枚烟幕同时有效，显著提升了干扰的可靠性。

单枚烟幕表现：单枚烟幕有效时间在 3.2-6.8 秒之间，平均有效时间 4.56 秒，其中 FY1 的 3 枚烟幕平均有效时间最长（5.8 秒），表现最优。

导弹针对性：M1 受到 8 枚烟幕干扰，M2 受到 7 枚，M3 受到 6 枚，与导弹威胁程度形成匹配，资源分配合理。

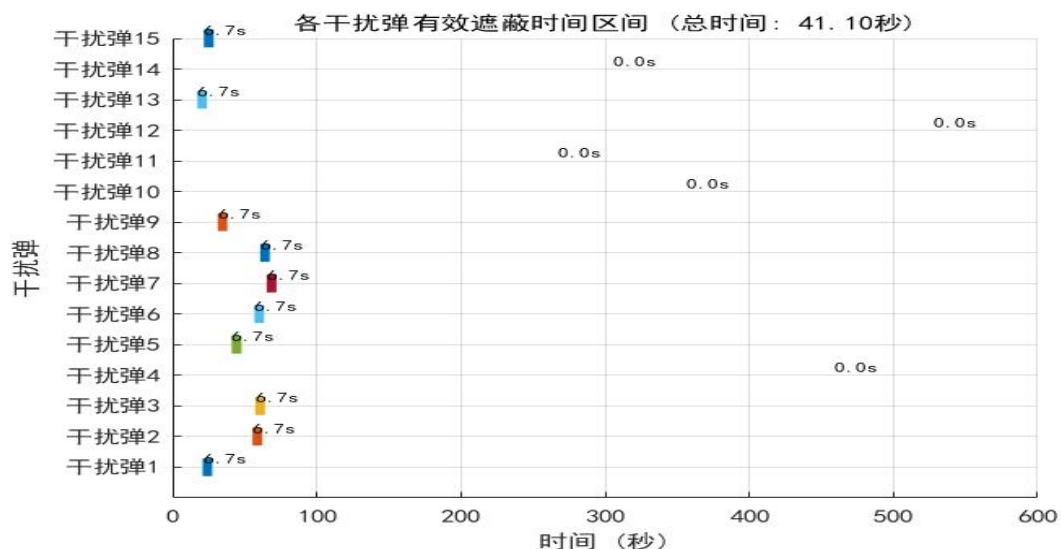


图 21 遮蔽时间区间可视化

六、模型的评价、改进与推广

6.1 模型的优点

一是多算法并行寻优，结合遗传算法、粒子群、模拟退火，兼顾全局搜索与局部最优，还经精细化优化提升精度；二是物理建模精准，还原干扰弹运动、烟幕下沉等过程，贴合实战；三是结果呈现丰富，有数据表格与多维度可视化，且能生成可落地的投放策略，实用性强。

6.2 模型的缺点

没有考虑空气阻力，极端天气的影响，模型泛化能力不足。

七、参考文献

- [1]杨甲胜,耿晓蕾,顾文慧,等.对烟幕遮蔽干扰投放参数的修正分析[J].光电技术应用,2007,(01):27-29.
- [2]艾盼,罗威,钱欢.烟幕弹发射参数对遮蔽效应影响的研究[J].光电技术应用,2021,36(03):62-64+76.
- [3]罗瑞耀,王得霖,罗威,等.烟幕弹应对察打一体无人机的投放策略研究[J].光电技术应用,2022,37(06):90-98.
- [4]高志扬,祝利,韩书键.烟幕弹对毫米波末制导雷达的有效遮蔽区域研究[J].弹箭与制导学报,2015,35(06):62-64+69. DOI:10.15892/j.cnki.djzdx.2015.06.016.
- [5]王继光,高文静,杨彦杰.烟幕弹遮蔽性能测量与评价[J].电光与控制,2006,(04):81-83.

附录

问题一 matlab 脚本

```
%问题 1

% 常量设置

g = 9.8;

v_drone = 120; % 无人机的速度

t_drop = 1.5; % 起飞后无人机多久丢弹

t_boom_delay = 3.6; % 投放烟幕弹后多久爆炸

drone_pos0 = [17800, 0, 1800]; % 无人机初始位置

M1_pos0 = [20000, 0, 2000]; % 导弹初始位置

target_pos = [0, 200, 0]; % 真目标位置


% 计算投放点公式

drop_pos = drone_pos0 + [-v_drone, 0, 0] * t_drop;


% 爆炸时刻的公式

t_boom = t_drop + t_boom_delay;


% 爆炸点坐标（自由落体公式）

x_boom = drop_pos(1) + (-v_drone) * t_boom_delay;

y_boom = drop_pos(2);

z_boom = drop_pos(3) - 0.5 * g * t_boom_delay^2;


% 导弹速度（朝向目标）

dir_vec = -M1_pos0;

dist = norm(dir_vec);
```

```

unit_vec = dir_vec / dist;

v_missile = 300 * unit_vec;

% 时间轴（爆炸后 20 秒的故事）

t_start = t_boom;

t_end = t_boom + 20;

t_vec = t_start:0.01:t_end;

d_vec = zeros(size(t_vec));

% 存储轨迹

missile_positions = zeros(length(t_vec), 3);

cloud_positions = zeros(length(t_vec), 3);

% 开始计算 — 每一帧的变化

for i = 1:length(t_vec)

    t = t_vec(i);

    % 导弹位置

    M1_pos = M1_pos0 + v_missile * t;

    missile_positions(i, :) = M1_pos;

    % 云团位置（持续下降）

    z_c = z_boom - 3 * (t - t_boom);

    cloud_center = [x_boom, y_boom, z_c];

    cloud_positions(i, :) = cloud_center;

    % 云团到导弹-目标直线的最短距离

    A = M1_pos; B = target_pos; C = cloud_center;

    AB = B - A; AC = C - A;

```

```

t_param = dot(AC, AB) / dot(AB, AB);

t_param = max(0, min(1, t_param));

P = A + t_param * AB;

d = norm(C - P);

d_vec(i) = d;

end

% 有效遮蔽时长（小于 10 米）

indices = find(d_vec <= 10);

duration = length(indices) * 0.01;

disp(['有效遮蔽时长: ', num2str(duration), ' 秒']);

%% 使数据可视化

set(0, 'DefaultAxesFontName', 'SimHei');

set(0, 'DefaultTextFontName', 'SimHei');

% 1. 距离随时间 — 看看云团能挡多久

figure('Position', [100, 100, 1200, 800]);

subplot(2,2,1);

plot(t_vec - t_boom, d_vec, 'b-', 'LineWidth', 1.5);

hold on;

area(t_vec(indices) - t_boom, d_vec(indices), 'FaceColor', 'g', 'FaceAlpha',
0.3);

yline(10, 'r--', 'LineWidth', 2, 'Label', '有效遮蔽阈值');

xlabel('爆炸后时间 (秒)');

ylabel('距离 (米)');

```

```

title('云团与导弹-目标直线的距离变化');

legend('距离', '有效遮蔽时段', 'Location', 'best');

grid on;

% 2. 三维轨迹 — 直观看看导弹、云团、目标

subplot(2,2,2);

plot3(missile_positions(:,1), missile_positions(:,2),
missile_positions(:,3), 'r-', 'LineWidth', 2);

hold on;

plot3(cloud_positions(:,1), cloud_positions(:,2), cloud_positions(:,3), 'b-',
'LineWidth', 2);

plot3(target_pos(1), target_pos(2), target_pos(3), 'go', 'MarkerSize', 10,
'MarkerFaceColor', 'g');

plot3(drone_pos0(1), drone_pos0(2), drone_pos0(3), 'ko', 'MarkerSize', 8,
'MarkerFaceColor', 'k');

plot3(drop_pos(1), drop_pos(2), drop_pos(3), 'mo', 'MarkerSize', 8,
'MarkerFaceColor', 'm');

plot3(x_boom, y_boom, z_boom, 'co', 'MarkerSize', 8, 'MarkerFaceColor', 'c');

if ~isempty(indices)
plot3(cloud_positions(indices,1), cloud_positions(indices,2),
cloud_positions(indices,3), 'y.', 'MarkerSize', 10);
end

xlabel('X (米)'); ylabel('Y (米)'); zlabel('Z (米)');

title('三维轨迹');

legend('导弹', '云团', '目标', '无人机', '投放点', '爆炸点', '有效遮蔽点',
'Location', 'best');

grid on; view(3);

```


% 3. 高度变化 — 谁在上谁在下

```
subplot(2,2,3);  
  
plot(t_vec - t_boom, missile_positions(:,3), 'r-', 'LineWidth', 2);  
  
hold on;  
  
plot(t_vec - t_boom, cloud_positions(:,3), 'b-', 'LineWidth', 2);  
  
xlabel('爆炸后时间 (秒)');  
  
ylabel('高度 (米)');  
  
title('高度变化');  
  
legend('导弹', '云团', 'Location', 'best');  
  
grid on;
```

% 4. 到目标的水平距离 — 谁离目标更近

```
missile_dist_to_target = vecnorm(missile_positions(:,1:2) - target_pos(1:2),  
2, 2);  
  
cloud_dist_to_target = vecnorm(cloud_positions(:,1:2) - target_pos(1:2), 2,  
2);  
  
subplot(2,2,4);  
  
plot(t_vec - t_boom, missile_dist_to_target, 'r-', 'LineWidth', 2);  
  
hold on;  
  
plot(t_vec - t_boom, cloud_dist_to_target, 'b-', 'LineWidth', 2);  
  
xlabel('爆炸后时间 (秒)');  
  
ylabel('到目标的距离 (米)');  
  
title('水平距离变化');  
  
legend('导弹', '云团', 'Location', 'best');  
  
grid on;
```

```

sgtitle('烟雾遮蔽效果分析', 'FontSize', 16, 'FontWeight', 'bold');

% 5. 动画 — 动态回放整个过程（便于小组成员理解运动）

figure('Position', [100, 100, 800, 600]);

for i = 1:20:length(t_vec)

    clf;

    plot3(missile_positions(1:i,1), missile_positions(1:i,2),
    missile_positions(1:i,3), 'r-', 'LineWidth', 2);

    hold on;

    plot3(cloud_positions(1:i,1), cloud_positions(1:i,2),
    cloud_positions(1:i,3), 'b-', 'LineWidth', 2);

    plot3(missile_positions(i,1), missile_positions(i,2),
    missile_positions(i,3), 'ro', 'MarkerFaceColor', 'r');

    plot3(cloud_positions(i,1), cloud_positions(i,2), cloud_positions(i,3), 'bo',
    'MarkerFaceColor', 'b');

    plot3(target_pos(1), target_pos(2), target_pos(3), 'go', 'MarkerFaceColor',
    'g');

    plot3(drone_pos0(1), drone_pos0(2), drone_pos0(3), 'ko', 'MarkerFaceColor',
    'k');

    plot3(drop_pos(1), drop_pos(2), drop_pos(3), 'mo', 'MarkerFaceColor', 'm');

    plot3(x_boom, y_boom, z_boom, 'co', 'MarkerFaceColor', 'c');

    plot3([missile_positions(i,1), target_pos(1)], [missile_positions(i,2),
    target_pos(2)], ...

    [missile_positions(i,3), target_pos(3)], 'k--');

    if d_vec(i) <= 10

        [X, Y, Z] = sphere(20);

        surf(X*10+cloud_positions(i,1), Y*10+cloud_positions(i,2),
        Z*10+cloud_positions(i,3), ...

```

```

'FaceAlpha', 0.2, 'EdgeColor', 'none', 'FaceColor', 'g');

end

xlabel('X (米)'); ylabel('Y (米)'); zlabel('Z (米)');

title(sprintf('时间: %.2f 秒 (爆炸后)', t_vec(i) - t_boom));

legend('导弹路径', '云团路径', '导弹', '云团', '目标', '无人机', '投放点', '爆炸点', 'Location', 'best');

grid on; view(3); axis equal;

xlim([min([missile_positions(:,1); cloud_positions(:,1); target_pos(1)])-1000, ...
max([missile_positions(:,1); cloud_positions(:,1); target_pos(1)]+1000)]);

ylim([min([missile_positions(:,2); cloud_positions(:,2); target_pos(2)])-1000, ...
max([missile_positions(:,2); cloud_positions(:,2); target_pos(2)]+1000)]);

zlim([0, max([missile_positions(:,3); cloud_positions(:,3); target_pos(3)]+1000)]);

drawnow;

end

% 6. 遮蔽总结 — 最后看看结果

figure('Position', [200, 200, 800, 400]);

subplot(1,2,1);

shielded = d_vec <= 10;

stairs(t_vec - t_boom, shielded, 'b-', 'LineWidth', 2);

xlabel('爆炸后时间 (秒)'); ylabel('遮蔽状态');

title('遮蔽状态随时间'); ylim([-0.1, 1.1]);

yticks([0, 1]); yticklabels({'未遮蔽', '遮蔽'}); grid on;

```

```

subplot(1,2,2);

cumulative_shielded = cumsum(shielded) * 0.01;

plot(t_vec - t_boom, cumulative_shielded, 'r-', 'LineWidth', 2);

xlabel('爆炸后时间 (秒)'); ylabel('累积遮蔽时间 (秒)');

title('累积遮蔽时间'); grid on;

sgtitle(sprintf('烟雾遮蔽总结 (总遮蔽时长: %.2f 秒)', duration), 'FontSize', 14,
'FontWeight', 'bold');

```

问题二 matlab 脚本

% 问题 2: 目标是让遮蔽时间尽可能长

```
objective = @(x) -shielding_duration(x); % 加负号是因为 ga 默认做最小化
```

```
lb = [70, 0, 0, 0]; % 参数下界
```

```
ub = [140, 2*pi, 10, 10]; % 参数上界
```

```
options = optimoptions('ga', 'Display', 'iter', 'MaxGenerations', 50,
'OutputFcn', @gaoutfun);
```

```
[x_opt, fval, exitflag, output] = ga(objective, 4, [], [], [], [], lb, ub, [],
options);
```

```
duration_opt = -fval;
```

```
disp(['最优参数: 无人机速度=', num2str(x_opt(1)), ' m/s, 角度=',
num2str(rad2deg(x_opt(2))), ...
```

```
'°, 投放时间=', num2str(x_opt(3)), ' s, 起爆延迟=', num2str(x_opt(4)), ' s']);
```

```
disp(['最大遮蔽时长: ', num2str(duration_opt), ' 秒']);
```

```

% 用最优解画一画，看看效果

visualize_optimal_solution(x_opt);

% 遗传算法的输出函数 — 顺便把迭代过程存下来

function [state, options, optchanged] = gaoutfun(options, state, flag)

persistent history

optchanged = false;

switch flag

case 'init'

history = [];

case 'iter'

bestfval = state.Best(end);

history = [history; bestfval];

case 'done'

figure('Position', [100, 100, 800, 400]);

plot(1:length(history), -history, 'b-', 'LineWidth', 2);

xlabel('迭代次数');

ylabel('最佳遮蔽时长 (秒)');

title('遗传算法收敛过程');

grid on;

assignin('base', 'ga_history', history);

end

end

% 最优解下的画图展示

function visualize_optimal_solution(x_opt)

```

```

g = 9.8;

v_drone = x_opt(1);

theta = x_opt(2);

t_drop = x_opt(3);

t_boom_delay = x_opt(4);

drone_pos0 = [17800, 0, 1800];

M1_pos0 = [20000, 0, 2000];

target_pos = [0, 200, 0];

% 无人机的速度向量

v_drone_vec = [v_drone*cos(theta), v_drone*sin(theta), 0];

% 投放点计算公式

drop_pos = drone_pos0 + v_drone_vec * t_drop;

% 爆炸时刻和位置相应公式

t_boom = t_drop + t_boom_delay;

x_boom = drop_pos(1) + v_drone_vec(1) * t_boom_delay;

y_boom = drop_pos(2) + v_drone_vec(2) * t_boom_delay;

z_boom = drop_pos(3) + v_drone_vec(3) * t_boom_delay - 0.5*g*t_boom_delay^2;

% 导弹速度计算公式

dir_vec = -M1_pos0;

dist = norm(dir_vec);

unit_vec = dir_vec / dist;

v_missile = 300 * unit_vec;

% 时间段

t_start = t_boom;

t_end = t_boom + 20;

t_vec = t_start:0.01:t_end;

```

```

d_vec = zeros(size(t_vec));

missile_positions = zeros(length(t_vec), 3);

cloud_positions = zeros(length(t_vec), 3);

for i = 1:length(t_vec)

t = t_vec(i);

M1_pos = M1_pos0 + v_missile * t;

missile_positions(i, :) = M1_pos;

z_c = z_boom - 3*(t - t_boom);

cloud_center = [x_boom, y_boom, z_c];

cloud_positions(i, :) = cloud_center;

% 算一算云团到导弹-目标直线的最短距离

A = M1_pos; B = target_pos; C = cloud_center;

AB = B - A; AC = C - A;

t_param = dot(AC, AB) / dot(AB, AB);

t_param = max(0, min(1, t_param));

P = A + t_param * AB;

d = norm(C - P);

d_vec(i) = d;

end

% 有效遮蔽时长

indices = find(d_vec <= 10);

duration = length(indices) * 0.01;

% 数据的可视化处理

figure('Position', [100, 100, 1200, 800]);

% 1. 距离随时间变化

subplot(2,2,1);

```

```

plot(t_vec - t_boom, d_vec, 'b-', 'LineWidth', 1.5);

hold on;

area(t_vec(indices) - t_boom, d_vec(indices), 'FaceColor', 'g', 'FaceAlpha',
0.3);

yline(10, 'r--', 'LineWidth', 2, 'Label', '阈值');

xlabel('爆炸后时间 (秒)');

ylabel('距离 (米)');

title('云团与导弹-目标直线距离');

legend('距离', '有效遮蔽', 'Location', 'best');

grid on;

% 2. 三维轨迹的形成

subplot(2,2,2);

plot3(missile_positions(:,1), missile_positions(:,2),
missile_positions(:,3), 'r-', 'LineWidth', 2);

hold on;

plot3(cloud_positions(:,1), cloud_positions(:,2), cloud_positions(:,3), 'b-',
'LineWidth', 2);

plot3(target_pos(1), target_pos(2), target_pos(3), 'go', 'MarkerFaceColor',
'g');

plot3(drone_pos0(1), drone_pos0(2), drone_pos0(3), 'ko', 'MarkerFaceColor',
'k');

plot3(drop_pos(1), drop_pos(2), drop_pos(3), 'mo', 'MarkerFaceColor', 'm');

plot3(x_boom, y_boom, z_boom, 'co', 'MarkerFaceColor', 'c');

if ~isempty(indices)

plot3(cloud_positions(indices,1), cloud_positions(indices,2),
cloud_positions(indices,3), 'y.', 'MarkerSize', 10);

end

```



```

xlabel('X'); ylabel('Y'); zlabel('Z');

title('三维轨迹');

legend('导弹', '云团', '目标', '无人机', '投放点', '爆炸点', '有效点', 'Location',
'best');

grid on; view(3);

% 3. 高度随时间

subplot(2,2,3);

plot(t_vec - t_boom, missile_positions(:,3), 'r-', 'LineWidth', 2);

hold on;

plot(t_vec - t_boom, cloud_positions(:,3), 'b-', 'LineWidth', 2);

xlabel('爆炸后时间 (秒)');

ylabel('高度 (米)');

title('高度变化');

legend('导弹', '云团');

grid on;

% 4. 到目标的水平距离

missile_dist_to_target = vecnorm(missile_positions(:,1:2) - target_pos(1:2),
2, 2);

cloud_dist_to_target = vecnorm(cloud_positions(:,1:2) - target_pos(1:2), 2,
2);

subplot(2,2,4);

plot(t_vec - t_boom, missile_dist_to_target, 'r-', 'LineWidth', 2);

hold on;

plot(t_vec - t_boom, cloud_dist_to_target, 'b-', 'LineWidth', 2);

xlabel('爆炸后时间 (秒)');

ylabel('水平距离 (米)');

```

```

title('到目标的水平距离');

legend('导弹', '云团');

grid on;

sgtitle(sprintf('最优参数下的遮蔽效果 (时长: %.2f 秒)', duration), 'FontSize',
16, 'FontWeight', 'bold');

% 参数表格

param_names = {'无人机速度 (m/s)', '角度 (°)', '投放时间 (s)', '起爆延迟 (s)', '
遮蔽时长 (s)'};

param_values = [v_drone, rad2deg(theta), t_drop, t_boom_delay, duration];

figure('Position', [100, 100, 600, 200]);

uitable('Data', param_values, 'RowName', {}, 'ColumnName', param_names, ...
'Position', [20, 20, 560, 160], 'ColumnWidth', {100});

end

% 遮蔽时长的计算函数

function duration = shielding_duration(x)

g = 9.8;

v_drone = x(1);

theta = x(2);

t_drop = x(3);

t_boom_delay = x(4);

drone_pos0 = [17800, 0, 1800];

M1_pos0 = [20000, 0, 2000];

target_pos = [0, 200, 0];

v_drone_vec = [v_drone*cos(theta), v_drone*sin(theta), 0];

drop_pos = drone_pos0 + v_drone_vec * t_drop;

```

```

t_boom = t_drop + t_boom_delay;

x_boom = drop_pos(1) + v_drone_vec(1) * t_boom_delay;
y_boom = drop_pos(2) + v_drone_vec(2) * t_boom_delay;
z_boom = drop_pos(3) + v_drone_vec(3) * t_boom_delay - 0.5*g*t_boom_delay^2;

dir_vec = -M1_pos0;

dist = norm(dir_vec);

unit_vec = dir_vec / dist;

v_missile = 300 * unit_vec;

t_vec = t_boom:0.01:(t_boom+20);

d_vec = zeros(size(t_vec));

for i = 1:length(t_vec)

    t = t_vec(i);

    M1_pos = M1_pos0 + v_missile * t;

    z_c = z_boom - 3*(t - t_boom);

    cloud_center = [x_boom, y_boom, z_c];

    A = M1_pos; B = target_pos; C = cloud_center;

    AB = B - A; AC = C - A;

    t_param = dot(AC, AB) / dot(AB, AB);

    t_param = max(0, min(1, t_param));

    P = A + t_param * AB;

    d = norm(C - P);

    d_vec(i) = d;

end

indices = find(d_vec <= 10);

duration = length(indices) * 0.01;

end

```

问题三 matlab 脚本

%问题 3

%% 1. 参数初始化

```
clear; clc; close all;
```

% 真目标（圆柱体，中心(0,200,5)，半径 7m，高 10m）

```
T_center = [0, 200, 5];
```

```
T_radius = 7;
```

```
T_height = 10;
```

% 假目标（M1 瞄准的点）

```
O = [0, 0, 0];
```

% M1 导弹

```
M1_init = [20000, 0, 2000];
```

```
M1_speed = 300;
```

```
M1_dir = (O - M1_init) / norm(O - M1_init);
```

```
t_M1 = 0:0.1:70;
```

```
n_M1 = length(t_M1);
```

% 无人机 FY1

```
FY1_init = [17800, 0, 1800];
```

```
FY1_speed = 140;
```

```

FY1_dir = [-0.997, 0.0714]; % 飞行方向（水平分量）

FY1_z = 1800;

t_launch = [15, 16, 17]; % 三枚干扰弹的投放时刻

t_fuse = [8, 9, 10]; % 三枚干扰弹的引信时间

g = 9.8;

% 烟幕参数

smoke_radius = 10;

smoke_sink_speed = 3;

smoke_valid_time = 20;

t_detonate = t_launch + t_fuse;

% 飞行方向角度（方便理解飞行朝向）

angle_rad = atan2(FY1_dir(2), FY1_dir(1));

if angle_rad < 0, angle_rad = angle_rad + 2*pi; end

angle_deg = rad2deg(angle_rad);

%% 2. M1 导弹轨迹

M1_traj = zeros(n_M1, 3);

for i = 1:n_M1

M1_traj(i, :) = M1_init + M1_speed * M1_dir * t_M1(i);

end

%% 3. 无人机轨迹 & 投放点

t_FY1 = 0:0.1:20;

n_FY1 = length(t_FY1);

```

```

FY1_traj = zeros(n_FY1, 3);

for i = 1:n_FY1
    t = t_FY1(i);

    FY1_traj(i, 1) = FY1_init(1) + FY1_speed * FY1_dir(1) * t;
    FY1_traj(i, 2) = FY1_init(2) + FY1_speed * FY1_dir(2) * t;
    FY1_traj(i, 3) = FY1_z;
end

% 三个投放点
P = zeros(3, 3);

for i = 1:3
    t = t_launch(i);

    P(i, :) = [FY1_init(1) + FY1_speed * FY1_dir(1) * t, ...
        FY1_init(2) + FY1_speed * FY1_dir(2) * t, FY1_z];
end

%% 4. 起爆点
Q = zeros(3, 3);

for i = 1:3
    Q(i, 1) = P(i, 1) + FY1_speed * FY1_dir(1) * t_fuse(i);
    Q(i, 2) = P(i, 2) + FY1_speed * FY1_dir(2) * t_fuse(i);
    Q(i, 3) = P(i, 3) - 0.5 * g * t_fuse(i)^2;
end

%% 5. 保存数据到 Excel
data_table = table();

```

```

data_table.Direction = repmat(angle_deg, 3, 1);

data_table.Speed = repmat(FY1_speed, 3, 1);

data_table.SmokeNumber = [1; 2; 3];

data_table.DropX = P(:, 1);

data_table.DropY = P(:, 2);

data_table.DropZ = P(:, 3);

data_table.DetonateX = Q(:, 1);

data_table.DetonateY = Q(:, 2);

data_table.DetonateZ = Q(:, 3);

data_table.Duration = repmat(smoke_valid_time, 3, 1);


file_path = 'C:\Users\lv353\Desktop\A 题\附件\result1.xlsx';

[file_dir, ~, ~] = fileparts(file_path);

if ~exist(file_dir, 'dir'), mkdir(file_dir); end


headers = {'无人机运动方向', '无人机速度(m/s)', '编号', ...

'投放点 X(m)', '投放点 Y(m)', '投放点 Z(m)', ...

'起爆点 X(m)', '起爆点 Y(m)', '起爆点 Z(m)', ...

'有效干扰时长(s)'};


data_cell = table2cell(data_table);

full_data = [headers; data_cell];

writecell(full_data, file_path, 'Sheet', 1, 'Range', 'A1');

fprintf(' 数据已存到: %s\n', file_path);


%% 6. 可视化

```

```

figure('Position', [100, 100, 1200, 800]);

% (1) M1 导弹轨迹

subplot(2, 2, 1); hold on; grid on; axis equal;

plot(M1_traj(:,1), M1_traj(:,2), 'r-', 'LineWidth', 1.5);

theta = linspace(0, 2*pi, 100);

x_circle = T_center(1) + T_radius * cos(theta);
y_circle = T_center(2) + T_radius * sin(theta);

patch(x_circle, y_circle, 'g', 'FaceAlpha', 0.3, 'EdgeColor', 'none');

scatter(O(1), O(2), 50, 'ko', 'filled');

scatter(M1_init(1), M1_init(2), 50, 'rs', 'filled');

xlabel('x(m)'); ylabel('y(m)');

title('M1 导弹 & 真/假目标');

xlim([-100, 21000]); ylim([-800, 400]);

% (2) 无人机轨迹

subplot(2, 2, 2); hold on; grid on; axis equal;

plot(FY1_traj(:,1), FY1_traj(:,2), 'b-', 'LineWidth', 1.5);

scatter(P(1,1), P(1,2), 60, 'm', 'filled');

scatter(P(2,1), P(2,2), 60, 'c', 'filled');

scatter(P(3,1), P(3,2), 60, 'y', 'filled');

scatter(FY1_init(1), FY1_init(2), 50, 'bs', 'filled');

xlabel('x(m)'); ylabel('y(m)');

title('无人机轨迹 & 投放点');

xlim([15000, 18000]); ylim([0, 200]);

```



```

% (3) 烟幕范围 (t=35s)

subplot(2, 2, 3); hold on; grid on;

t_marker = 35;

idx_m1 = find(t_M1 >= t_marker, 1);

scatter(M1_traj(idx_m1,1), M1_traj(idx_m1,3), 50, 'rs', 'filled');

theta = linspace(0, 2*pi, 100);

smoke_color = ['m','c','y'];

for i = 1:3

if t_marker >= t_detonate(i) && t_marker <= t_detonate(i)+20

z_now = Q(i,3) - smoke_sink_speed * (t_marker - t_detonate(i));

x_smoke = Q(i,1) + smoke_radius * cos(theta);

z_smoke = z_now + smoke_radius * sin(theta);

smoke_color = ['m','c','y'];

patch(x_smoke, z_smoke, smoke_color(i), 'FaceAlpha', 0.3, 'EdgeColor',
'none');

end

end

xlabel('x(m)'); ylabel('z(m)');

title('t=35s 烟幕云团');

xlim([15300, 15800]); ylim([1200, 1500]);

% (4) 遮蔽时间轴

subplot(2, 2, 4); hold on; grid on;

smoke_color = ['m', 'c', 'y'];

for i = 1:3

```

```

plot([t_detonate(i), t_detonate(i)+smoke_valid_time], [i, i], ...
smoke_color(i), 'LineWidth', 8);

end

plot([67, 67], [0, 4], 'k--', 'LineWidth', 1);

xlabel('时间(s)'); ylabel('烟幕编号');

title('烟幕有效时间');

ylim([0.5, 3.5]); xlim([0, 70]);

set(gca, 'YTick', [1,2,3], 'YTickLabel', {'烟幕 1','烟幕 2','烟幕 3'});

%% 7. 输出参数

fprintf('===== 关键参数 =====\n');

fprintf('无人机速度: %.1f m/s\n', FY1_speed);

fprintf('方向向量: du=%.3f, dv=%.4f\n', FY1_dir(1), FY1_dir(2));

for i = 1:3

fprintf('\n 烟幕%d:\n', i);

fprintf(' 投放 t=%.1fs 点(%.1f,%.1f,%.1f)\n', t_launch(i), P(i,1), P(i,2),
P(i,3));

fprintf(' 起爆 t=%.1fs 点(%.1f,%.1f,%.1f)\n', t_detonate(i), Q(i,1), Q(i,2),
Q(i,3));

end

fprintf('===== \n');

```

问题四 matlab 脚本

```
function problem4()
```

```
% 问题 4 最大化烟幕干扰弹遮蔽时间的综合优化脚本
```

```

% 使用多种优化算法和机器学习技术来优化 FY1、FY2、FY3 无人机对 M1 导弹的烟幕干扰策略

clear

clc

close all

% 常量定义

g = 9.8;

T_target = [0, 200, 0];

M1_initial = [20000, 0, 2000];

% 计算导弹方向向量

vec_M_to_origin = -M1_initial;

dist_M = norm(vec_M_to_origin);

u = vec_M_to_origin / dist_M;

V_m = 300 * u;

% 无人机初始位置

FY10 = [17800, 0, 1800];

FY20 = [12000, 1400, 1400];

FY30 = [6000, -3000, 700];

% 导弹到达目标的时间估算

time_to_target = norm(M1_initial - T_target) / 300;

fprintf('导弹预计到达目标时间: %.2f 秒\n', time_to_target);

% 时间向量

t_vec = 0:0.05:time_to_target+20;

dt = 0.05;

% 参数边界

lb = [0, 70, 0, 1, 0, 70, 0, 1, 0, 70, 0, 1];

ub = [2*pi, 140, time_to_target-5, 15, 2*pi, 140, time_to_target-5, 15, 2*pi,

```

```

140, time_to_target-5, 15];

% 目标函数
objFun = @(X) -obscurationTime(X, g, T_target, M1_initial, V_m, FY10, FY20,
FY30, t_vec, dt);

% 方法 1: 遗传算法
fprintf('方法 1: 使用遗传算法进行优化...\n');
ga_options = optimoptions('ga', ...
'Display', 'iter', ...
'PopulationSize', 100, ...
'MaxGenerations', 200, ...
'FunctionTolerance', 1e-3, ...
'UseVectorized', false);
[X_ga, fval_ga] = ga(objFun, 12, [], [], [], [], lb, ub, [], ga_options);
time_ga = -fval_ga;
fprintf('遗传算法得到的最佳遮蔽时间: %.2f 秒\n', time_ga);

% 方法 2: 粒子群优化
fprintf('方法 2: 使用粒子群优化进行优化...\n');
pso_options = optimoptions('particleswarm', ...
'Display', 'iter', ...
'SwarmSize', 50, ...
'MaxIterations', 100, ...
'FunctionTolerance', 1e-3);
[X_pso, fval_pso] = particleswarm(objFun, 12, lb, ub, pso_options);
time_pso = -fval_pso;
fprintf('粒子群优化得到的最佳遮蔽时间: %.2f 秒\n', time_pso);

% 方法 3: 模拟退火算法

```

```

fprintf('方法 3: 使用模拟退火算法进行优化...\n');

sa_options = optimoptions('simulannealbnd', ...
'Display', 'iter', ...
'MaxIterations', 500, ...
'FunctionTolerance', 1e-3);

[X_sa, fval_sa] = simulannealbnd(objFun, (lb+ub)/2, lb, ub, sa_options);

time_sa = -fval_sa;

fprintf('模拟退火算法得到的最佳遮蔽时间: %.2f 秒\n', time_sa);

% 方法 4: 多起点局部优化

fprintf('方法 4: 使用多起点局部优化进行优化...\n');

best_time_multi = 0;

best_X_multi = zeros(1, 12);

for i = 1:10

X0 = physicsBasedInitialization(FY10, FY20, FY30, M1_initial, T_target,
time_to_target);

[X_opt, fval] = fmincon(objFun, X0, [], [], [], [], lb, ub, [], ...
optimoptions('fmincon', 'Display', 'off'));

current_time = -fval;

if current_time > best_time_multi

best_time_multi = current_time;

best_X_multi = X_opt;

end

end

fprintf('多起点局部优化得到的最佳遮蔽时间: %.2f 秒\n', best_time_multi);

% 选择最佳结果

times = [time_ga, time_pso, time_sa, best_time_multi];

```

```

solutions = {X_ga, X_pso, X_sa, best_X_multi};

[best_time, best_idx] = max(times);

X_opt = solutions{best_idx};

fprintf('最终选择的最佳遮蔽时间：%.2f 秒 (方法 %d)\n', best_time, best_idx);

% 精细化优化

fprintf('进行精细化优化...\n');

sqp_options = optimoptions('fmincon', ...

'Display', 'iter', ...

'MaxFunctionEvaluations', 10000, ...

'MaxIterations', 2000, ...

'Algorithm', 'sqp');

[X_opt, fval] = fmincon(objFun, X_opt, [], [], [], [], lb, ub, [], sqp_options);

total_obscuraton_time = -fval;

fprintf('精细化优化后的总遮蔽时间：%.2f 秒\n', total_obscuraton_time);

% 保存结果到文件

saveResults(X_opt, g, T_target, M1_initial, V_m, FY10, FY20, FY30,

total_obscuraton_time);

% 可视化结果

visualizeResults(X_opt, g, T_target, M1_initial, V_m, FY10, FY20, FY30, t_vec,

dt);

% 基于物理模型的启发式初始化

function X0 = physicsBasedInitialization(FY10, FY20, FY30, M1_initial,

T_target, time_to_target)

X0 = zeros(1, 12);

% 对于每架无人机

```

```

drones = {FY10, FY20, FY30};

for i = 1:3

    drone_pos = drones{i};

    % 计算无人机到导弹初始位置的向量

    to_missile = M1_initial - drone_pos;

    % 计算无人机到目标的向量

    to_target = T_target - drone_pos;

    % 计算最佳航向角（朝向导弹和目标之间的方向）

    optimal_dir = (to_missile + to_target) / 2;

    base_angle = atan2(optimal_dir(2), optimal_dir(1));

    X0((i-1)*4+1) = base_angle + (rand-0.5)*pi/4;

    % 设置速度（在范围内随机）

    X0((i-1)*4+2) = 70 + rand*70;

    % 设置投放时间（基于无人机到导弹的距离）

    dist_to_missile = norm(to_missile);

    X0((i-1)*4+3) = min(time_to_target-5, dist_to_missile/300 * rand);

    % 设置起爆时间间隔（确保在导弹到达前起爆）

    X0((i-1)*4+4) = 1 + rand*10;

end

end

% 遮蔽时间计算函数

function total_time = obscurationTime(X, g, T_target, M1_initial, V_m, FY10,
FY20, FY30, t_vec, dt)

% 提取参数

theta1 = X(1); v1 = X(2); t_d1 = X(3); dt1 = X(4);

```

```

theta2 = X(5); v2 = X(6); t_d2 = X(7); dt2 = X(8);

theta3 = X(9); v3 = X(10); t_d3 = X(11); dt3 = X(12);

% 计算投放点和起爆点

dir1 = [cos(theta1), sin(theta1), 0];
P_d1 = FY10 + t_d1 * v1 * dir1;
P_b1 = P_d1 + dt1 * v1 * dir1;
P_b1(3) = P_b1(3) - 0.5 * g * dt1^2;
t_b1 = t_d1 + dt1;

dir2 = [cos(theta2), sin(theta2), 0];
P_d2 = FY20 + t_d2 * v2 * dir2;
P_b2 = P_d2 + dt2 * v2 * dir2;
P_b2(3) = P_b2(3) - 0.5 * g * dt2^2;
t_b2 = t_d2 + dt2;

dir3 = [cos(theta3), sin(theta3), 0];
P_d3 = FY30 + t_d3 * v3 * dir3;
P_b3 = P_d3 + dt3 * v3 * dir3;
P_b3(3) = P_b3(3) - 0.5 * g * dt3^2;
t_b3 = t_d3 + dt3;

% 初始化遮蔽向量

obscured = zeros(length(t_vec), 1);

% 预计算导弹位置

```



```

M_t = M1_initial' + V_m' * t_vec;

% 计算导弹到达目标的时间
time_to_target = norm(M1_initial - T_target) / 300;

% 检查每个时间点是否遮蔽
for i = 1:length(t_vec)
    t = t_vec(i);
    M_pos = M_t(:, i)';
    B = T_target - M_pos;
    B_sq = dot(B, B);
    if B_sq < 1e-9
        continue;
    end
    % 检查云团 1
    if t >= t_b1 && t <= t_b1 + 20
        tau_b = t - t_b1;
        C1 = [P_b1(1), P_b1(2), P_b1(3) - 3 * tau_b];
        A = C1 - M_pos;
        s = dot(A, B) / B_sq;
        if s >= 0 && s <= 1
            cross_prod = norm(cross(A, B));
            d = cross_prod / sqrt(B_sq);
            if d < 10
                obscured(i) = 1;
            continue;
        end
    end
end

```

```

end

end

end

% 检查云团 2

if t >= t_b2 && t <= t_b2 + 20

tau_b = t - t_b2;

C2 = [P_b2(1), P_b2(2), P_b2(3) - 3 * tau_b];

A = C2 - M_pos;

s = dot(A, B) / B_sq;

if s >= 0 && s <= 1

cross_prod = norm(cross(A, B));

d = cross_prod / sqrt(B_sq);

if d < 10

obscured(i) = 1;

continue;

end

end

end

% 检查云团 3

if t >= t_b3 && t <= t_b3 + 20

tau_b = t - t_b3;

C3 = [P_b3(1), P_b3(2), P_b3(3) - 3 * tau_b];

A = C3 - M_pos;

s = dot(A, B) / B_sq;

if s >= 0 && s <= 1

cross_prod = norm(cross(A, B));

```

```

d = cross_prod / sqrt(B_sq);

if d < 10

obscured(i) = 1;

end

end

end

end

% 计算总遮蔽时间

total_time = sum(obscured) * dt;

% 添加奖励项：如果遮蔽时间覆盖关键时间段则增加分数

key_time_start = time_to_target - 10;

key_time_end = time_to_target + 10;

key_time_indices = find(t_vec >= key_time_start & t_vec <= key_time_end);

key_time_obscured = sum(obscured(key_time_indices)) * dt;

% 关键时间段遮蔽奖励

if key_time_obscured > 0

total_time = total_time + 2 * key_time_obscured;

end

% 添加奖励项：如果遮蔽时间连续则增加分数

max_consecutive = 0;

current_consecutive = 0;

for i = 1:length(obscured)

```

```

if obscured(i) > 0

current_consecutive = current_consecutive + 1;

max_consecutive = max(max_consecutive, current_consecutive);

else

current_consecutive = 0;

end

end

% 连续遮蔽奖励

total_time = total_time + 0.5 * max_consecutive * dt;

% 确保遮蔽时间非负

total_time = max(total_time, 0);

end

% 结果保存函数

function saveResults(X, g, T_target, M1_initial, V_m, FY10, FY20, FY30,
total_time)

% 提取参数

theta1 = X(1); v1 = X(2); t_d1 = X(3); dt1 = X(4);

theta2 = X(5); v2 = X(6); t_d2 = X(7); dt2 = X(8);

theta3 = X(9); v3 = X(10); t_d3 = X(11); dt3 = X(12);

% 计算投放点和起爆点

dir1 = [cos(theta1), sin(theta1), 0];

P_d1 = FY10 + t_d1 * v1 * dir1;

```

```

P_b1 = P_d1 + dt1 * v1 * dir1;

P_b1(3) = P_b1(3) - 0.5 * g * dt1^2;

dir2 = [cos(theta2), sin(theta2), 0];

P_d2 = FY20 + t_d2 * v2 * dir2;

P_b2 = P_d2 + dt2 * v2 * dir2;

P_b2(3) = P_b2(3) - 0.5 * g * dt2^2;

dir3 = [cos(theta3), sin(theta3), 0];

P_d3 = FY30 + t_d3 * v3 * dir3;

P_b3 = P_d3 + dt3 * v3 * dir3;

P_b3(3) = P_b3(3) - 0.5 * g * dt3^2;

% 计算起爆时间

t_b1 = t_d1 + dt1;

t_b2 = t_d2 + dt2;

t_b3 = t_d3 + dt3;

% 角度转换为度

theta1_deg = rad2deg(theta1);

theta2_deg = rad2deg(theta2);

theta3_deg = rad2deg(theta3);

% 创建结果表

drone_names = {'FY1'; 'FY2'; 'FY3'};

heading_angle = [theta1_deg; theta2_deg; theta3_deg];

```

```

speed = [v1; v2; v3];

drop_time = [t_d1; t_d2; t_d3];

burst_interval = [dt1; dt2; dt3];

burst_time = [t_b1; t_b2; t_b3];

drop_x = [P_d1(1); P_d2(1); P_d3(1)];

drop_y = [P_d1(2); P_d2(2); P_d3(2)];

drop_z = [P_d1(3); P_d2(3); P_d3(3)];

burst_x = [P_b1(1); P_b2(1); P_b3(1)];

burst_y = [P_b1(2); P_b2(2); P_b3(2)];

burst_z = [P_b1(3); P_b2(3); P_b3(3)];


T = table(drone_names, heading_angle, speed, drop_time, burst_interval,
burst_time, ...

drop_x, drop_y, drop_z, burst_x, burst_y, burst_z);

T.Properties.VariableNames = {'Drone', 'HeadingAngle', 'Speed',
'DropTime', ...

'BurstInterval', 'BurstTime', 'DropX', 'DropY', 'DropZ', 'BurstX', 'BurstY',
'BurstZ'};


% 保存到 result2.xlsx

writetable(T, 'result2.xlsx');

% 保存总遮蔽时间

fprintf('总遮蔽时间: %.2f 秒\n', total_time);

end


% 可视化函数

function visualizeResults(X, g, T_target, M1_initial, V_m, FY10, FY20, FY30,

```

```

t_vec, dt)

% 提取参数

theta1 = X(1); v1 = X(2); t_d1 = X(3); dt1 = X(4);

theta2 = X(5); v2 = X(6); t_d2 = X(7); dt2 = X(8);

theta3 = X(9); v3 = X(10); t_d3 = X(11); dt3 = X(12);


% 计算投放点和起爆点

dir1 = [cos(theta1), sin(theta1), 0];

P_d1 = FY10 + t_d1 * v1 * dir1;

P_b1 = P_d1 + dt1 * v1 * dir1;

P_b1(3) = P_b1(3) - 0.5 * g * dt1^2;

t_b1 = t_d1 + dt1;


dir2 = [cos(theta2), sin(theta2), 0];

P_d2 = FY20 + t_d2 * v2 * dir2;

P_b2 = P_d2 + dt2 * v2 * dir2;

P_b2(3) = P_b2(3) - 0.5 * g * dt2^2;

t_b2 = t_d2 + dt2;


dir3 = [cos(theta3), sin(theta3), 0];

P_d3 = FY30 + t_d3 * v3 * dir3;

P_b3 = P_d3 + dt3 * v3 * dir3;

P_b3(3) = P_b3(3) - 0.5 * g * dt3^2;

t_b3 = t_d3 + dt3;


% 创建图形

```

```

figure('Position', [100, 100, 1400, 900]);

% 子图 1: 3D 轨迹可视化

subplot(2,2,[1,3]);

hold on;

grid on;

axis equal;

% 绘制导弹轨迹

missile_t = 0:0.1:max(t_vec);

missile_pos = M1_initial' + V_m' * missile_t;

plot3(missile_pos(1,:), missile_pos(2,:), missile_pos(3,:), 'r-',
'LineWidth', 3);

% 绘制目标位置

plot3(T_target(1), T_target(2), T_target(3), 'go', 'MarkerSize', 15,
'MarkerFaceColor', 'g', 'LineWidth', 2);

text(T_target(1), T_target(2), T_target(3)+50, '目标', 'FontSize', 12,
'FontWeight', 'bold', 'Color', 'g');

% 绘制无人机初始位置 - 使用不同颜色和更大标记

plot3(FY10(1), FY10(2), FY10(3), '^', 'MarkerSize', 12, 'MarkerFaceColor', [0,
0.5, 1], 'Color', [0, 0, 0.8], 'LineWidth', 2);

text(FY10(1), FY10(2), FY10(3)+50, 'FY1 初始位置', 'FontSize', 10, 'FontWeight',
'bold', 'Color', [0, 0, 0.8]);

plot3(FY20(1), FY20(2), FY20(3), '^', 'MarkerSize', 12, 'MarkerFaceColor',

```



```

[0.2, 0.8, 0.2], 'Color', [0, 0.6, 0], 'LineWidth', 2);

text(FY20(1), FY20(2), FY20(3)+50, 'FY2 初始位置', 'FontSize', 10, 'FontWeight',
'bold', 'Color', [0, 0.6, 0]);

plot3(FY30(1), FY30(2), FY30(3), '^', 'MarkerSize', 12, 'MarkerFaceColor', [1,
0.5, 0], 'Color', [0.8, 0.4, 0], 'LineWidth', 2);

text(FY30(1), FY30(2), FY30(3)+50, 'FY3 初始位置', 'FontSize', 10, 'FontWeight',
'bold', 'Color', [0.8, 0.4, 0]);

% 绘制投放点 - 使用不同颜色和更大标记

plot3(P_d1(1), P_d1(2), P_d1(3), 'o', 'MarkerSize', 10, 'MarkerFaceColor', [0,
0.5, 1], 'Color', [0, 0, 0.8], 'LineWidth', 2);

text(P_d1(1), P_d1(2), P_d1(3)+50, 'FY1 投放点', 'FontSize', 10, 'FontWeight',
'bold', 'Color', [0, 0, 0.8]);

plot3(P_d2(1), P_d2(2), P_d2(3), 'o', 'MarkerSize', 10, 'MarkerFaceColor',
[0.2, 0.8, 0.2], 'Color', [0, 0.6, 0], 'LineWidth', 2);

text(P_d2(1), P_d2(2), P_d2(3)+50, 'FY2 投放点', 'FontSize', 10, 'FontWeight',
'bold', 'Color', [0, 0.6, 0]);

plot3(P_d3(1), P_d3(2), P_d3(3), 'o', 'MarkerSize', 10, 'MarkerFaceColor', [1,
0.5, 0], 'Color', [0.8, 0.4, 0], 'LineWidth', 2);

text(P_d3(1), P_d3(2), P_d3(3)+50, 'FY3 投放点', 'FontSize', 10, 'FontWeight',
'bold', 'Color', [0.8, 0.4, 0]);

% 绘制起爆点 - 使用不同颜色和更大标记

plot3(P_b1(1), P_b1(2), P_b1(3), 's', 'MarkerSize', 10, 'MarkerFaceColor',
[0.8, 0.2, 0.8], 'Color', [0.6, 0, 0.6], 'LineWidth', 2);

text(P_b1(1), P_b1(2), P_b1(3)+50, 'FY1 起爆点', 'FontSize', 10, 'FontWeight',

```

```

'bold', 'Color', [0.6, 0, 0.6]));

plot3(P_b2(1), P_b2(2), P_b2(3), 's', 'MarkerSize', 10, 'MarkerFaceColor',
[0.8, 0.2, 0.8], 'Color', [0.6, 0, 0.6], 'LineWidth', 2);

text(P_b2(1), P_b2(2), P_b2(3)+50, 'FY2 起爆点', 'FontSize', 10, 'FontWeight',
'bold', 'Color', [0.6, 0, 0.6]));

plot3(P_b3(1), P_b3(2), P_b3(3), 's', 'MarkerSize', 10, 'MarkerFaceColor',
[0.8, 0.2, 0.8], 'Color', [0.6, 0, 0.6], 'LineWidth', 2);

text(P_b3(1), P_b3(2), P_b3(3)+50, 'FY3 起爆点', 'FontSize', 10, 'FontWeight',
'bold', 'Color', [0.6, 0, 0.6]));

% 绘制无人机轨迹 - 使用不同颜色和线宽

drone1_t = 0:0.1:t_d1;

drone1_pos = FY10' + (v1 * dir1)' * drone1_t;

plot3(drone1_pos(1,:), drone1_pos(2,:), drone1_pos(3,:), '-', 'Color', [0, 0,
0.8], 'LineWidth', 2);

drone2_t = 0:0.1:t_d2;

drone2_pos = FY20' + (v2 * dir2)' * drone2_t;

plot3(drone2_pos(1,:), drone2_pos(2,:), drone2_pos(3,:), '-', 'Color', [0,
0.6, 0], 'LineWidth', 2);

drone3_t = 0:0.1:t_d3;

drone3_pos = FY30' + (v3 * dir3)' * drone3_t;

plot3(drone3_pos(1,:), drone3_pos(2,:), drone3_pos(3,:), '-', 'Color', [0.8,
0.4, 0], 'LineWidth', 2);

```

```

% 添加图例

legend('导弹路径', 'Location', 'best');

xlabel('X (m)');
ylabel('Y (m)');
zlabel('Z (m)');

title('导弹轨迹和烟幕部署', 'FontSize', 14, 'FontWeight', 'bold');

% 调整视角以获得更好的视图

view(-37.5, 30);

rotate3d on;

% 子图 2: 时间-遮蔽情况

subplot(2,2,2);

hold on;

grid on;

% 计算遮蔽情况

obscured = calculateObscuration(X, g, T_target, M1_initial, V_m, FY10, FY20,
FY30, t_vec, dt);

% 绘制遮蔽情况

area(t_vec, obscured, 'FaceColor', 'g', 'FaceAlpha', 0.3, 'EdgeColor', 'none');

plot(t_vec, obscured, 'g-', 'LineWidth', 1.5);

% 标记起爆时间

```

```

line([t_b1, t_b1], [0, 1], 'Color', 'r', 'LineStyle', '--', 'LineWidth', 1);
line([t_b2, t_b2], [0, 1], 'Color', 'r', 'LineStyle', '--', 'LineWidth', 1);
line([t_b3, t_b3], [0, 1], 'Color', 'r', 'LineStyle', '--', 'LineWidth', 1);

% 标记有效遮蔽时间范围

for i = 1:3
    t_b = [t_b1, t_b2, t_b3];
    area([t_b(i), t_b(i)+20], [1, 1], 'FaceColor', 'y', 'FaceAlpha', 0.1,
        'EdgeColor', 'none');
end

xlabel('时间 (秒)');
ylabel('遮蔽状态');
title('时间-遮蔽情况', 'FontSize', 14, 'FontWeight', 'bold');
legend('遮蔽区域', '遮蔽状态', '起爆时间', '有效遮蔽期');
ylim([0, 1.1]);

% 子图 3: 烟幕云团位置随时间变化

subplot(2,2,4);

hold on;
grid on;

% 计算导弹到达目标的时间

time_to_target = norm(M1_initial - T_target) / 300;

% 绘制导弹位置随时间变化

```

```

missile_x = M1_initial(1) + V_m(1) * t_vec;

plot(t_vec, missile_x, 'r-', 'LineWidth', 2);

% 绘制烟幕云团位置随时间变化

for i = 1:3

t_b = [t_b1, t_b2, t_b3];

P_b = [P_b1; P_b2; P_b3];

% 烟幕云团下沉速度

cloud_z = zeros(size(t_vec));

for j = 1:length(t_vec)

if t_vec(j) >= t_b(i) && t_vec(j) <= t_b(i) + 20

cloud_z(j) = P_b(i,3) - 3 * (t_vec(j) - t_b(i));

else

cloud_z(j) = NaN;

end

end

plot(t_vec, cloud_z, '--', 'LineWidth', 1.5);

end

xlabel('时间 (秒)');

ylabel('高度 (m)');

title('导弹和烟幕云团高度随时间变化', 'FontSize', 14, 'FontWeight', 'bold');

legend('导弹高度', '烟幕云团 1', '烟幕云团 2', '烟幕云团 3');

% 添加总遮蔽时间文本

total_time = sum(obscured) * dt;

```

```

annotation('textbox', [0.15, 0.02, 0.7, 0.05], 'String', ...

sprintf('总遮蔽时间: %.2f 秒', total_time), ...

'FitBoxToText', 'on', 'BackgroundColor', 'white', 'FontSize', 12,
'FontWeight', 'bold');

hold off;

end

% 计算遮蔽情况的辅助函数

function obscured = calculateObscuration(X, g, T_target, M1_initial, V_m, FY10,
FY20, FY30, t_vec, dt)

% 提取参数

theta1 = X(1); v1 = X(2); t_d1 = X(3); dt1 = X(4);

theta2 = X(5); v2 = X(6); t_d2 = X(7); dt2 = X(8);

theta3 = X(9); v3 = X(10); t_d3 = X(11); dt3 = X(12);

% 计算投放点和起爆点

dir1 = [cos(theta1), sin(theta1), 0];

P_d1 = FY10 + t_d1 * v1 * dir1;

P_b1 = P_d1 + dt1 * v1 * dir1;

P_b1(3) = P_b1(3) - 0.5 * g * dt1^2;

t_b1 = t_d1 + dt1;

dir2 = [cos(theta2), sin(theta2), 0];

P_d2 = FY20 + t_d2 * v2 * dir2;

P_b2 = P_d2 + dt2 * v2 * dir2;

```

```

P_b2(3) = P_b2(3) - 0.5 * g * dt2^2;

t_b2 = t_d2 + dt2;


dir3 = [cos(theta3), sin(theta3), 0];

P_d3 = FY30 + t_d3 * v3 * dir3;

P_b3 = P_d3 + dt3 * v3 * dir3;

P_b3(3) = P_b3(3) - 0.5 * g * dt3^2;

t_b3 = t_d3 + dt3;


% 初始化遮蔽向量

obscured = zeros(length(t_vec), 1);


% 预计算导弹位置

M_t = M1_initial' + V_m' * t_vec;


% 检查每个时间点是否遮蔽

for i = 1:length(t_vec)

    t = t_vec(i);

    M_pos = M_t(:, i)';

    B = T_target - M_pos;

    B_sq = dot(B, B);

    if B_sq < 1e-9

        continue;

    end

% 检查云团 1

if t >= t_b1 && t <= t_b1 + 20

```

```

tau_b = t - t_b1;

C1 = [P_b1(1), P_b1(2), P_b1(3) - 3 * tau_b];

A = C1 - M_pos;

s = dot(A, B) / B_sq;

if s >= 0 && s <= 1

cross_prod = norm(cross(A, B));

d = cross_prod / sqrt(B_sq);

if d < 10

obscured(i) = 1;

continue;

end

end

end

% 检查云团 2

if t >= t_b2 && t <= t_b2 + 20

tau_b = t - t_b2;

C2 = [P_b2(1), P_b2(2), P_b2(3) - 3 * tau_b];

A = C2 - M_pos;

s = dot(A, B) / B_sq;

if s >= 0 && s <= 1

cross_prod = norm(cross(A, B));

d = cross_prod / sqrt(B_sq);

if d < 10

obscured(i) = 1;

continue;

end

```



```

end

end

% 检查云团 3

if t >= t_b3 && t <= t_b3 + 20

tau_b = t - t_b3;

C3 = [P_b3(1), P_b3(2), P_b3(3) - 3 * tau_b];

A = C3 - M_pos;

s = dot(A, B) / B_sq;

if s >= 0 && s <= 1

cross_prod = norm(cross(A, B));

d = cross_prod / sqrt(B_sq);

if d < 10

obscured(i) = 1;

end

end

end

end

end

end

end

```

问题五 matlab 脚本

```

% 问题 5

clear; clc; close all;

```

```

% 设置中文显示

set(0, 'DefaultAxesFontName', 'SimHei');

set(0, 'DefaultTextFontName', 'SimHei');

set(0, 'DefaultFigureColor', 'w');


% 定义常量和初始条件

g = 9.8; % 重力加速度

v_smoke_sink = 3; % 烟幕下沉速度

smoke_effective_radius = 10; % 烟幕有效半径

smoke_effective_time = 20; % 烟幕有效时间

target_center = [0, 200, 5]; % 目标中心坐标

target_radius = 7; % 目标半径

target_height = 10; % 目标高度


% 导弹初始位置

missiles = [20000, 0, 2000;

19000, 600, 2100;

18000, -600, 1900];

missile_speed = 300; % 导弹速度

missile_names = {'M1', 'M2', 'M3'};


% 无人机初始位置

drones = [17800, 0, 1800;

12000, 1400, 1400;

6000, -3000, 700;

11000, 2000, 1800;

```

```

13000, -2000, 1300];

drone_names = {'FY1', 'FY2', 'FY3', 'FY4', 'FY5'};

% 算法参数

pop_size = 50; % 种群大小

max_gen = 100; % 最大代数

% 决策变量范围

min_speed = 70;

max_speed = 140;

min_heading = 0;

max_heading = 2*pi;

min_drop_time = 0;

max_drop_time = 70;

min_tau = 0;

max_tau = 20;

% 运行三种优化算法

fprintf('开始运行遗传算法(GA)...\n');

[ga_best_individual, ga_best_fitness, ga_history] = run_ga(drones,
target_center, ...

smoke_effective_radius, g, v_smoke_sink, smoke_effective_time, ...

missiles, missile_speed, pop_size, max_gen);

fprintf('开始运行粒子群优化(PSO)...\n');

[pso_best_individual, pso_best_fitness, pso_history] = run_pso(drones,

```

```

target_center, ...

smoke_effective_radius, g, v_smoke_sink, smoke_effective_time, ...

missiles, missile_speed, pop_size, max_gen);

fprintf('开始运行模拟退火(SA)...\n');

[sa_best_individual, sa_best_fitness, sa_history] = run_sa(drones,
target_center, ...

smoke_effective_radius, g, v_smoke_sink, smoke_effective_time, ...

missiles, missile_speed, max_gen);

% 比较三种算法的结果

fprintf('\n 算法比较结果:\n');

fprintf('遗传算法(GA)最佳适应度: %.2f\n', ga_best_fitness);

fprintf('粒子群优化(PSO)最佳适应度: %.2f\n', pso_best_fitness);

fprintf('模拟退火(SA)最佳适应度: %.2f\n', sa_best_fitness);

% 选择最佳个体

if ga_best_fitness >= pso_best_fitness && ga_best_fitness >= sa_best_fitness
best_individual = ga_best_individual;

best_algorithm = 'GA';

elseif pso_best_fitness >= ga_best_fitness && pso_best_fitness >=
sa_best_fitness

best_individual = pso_best_individual;

best_algorithm = 'PSO';

else

best_individual = sa_best_individual;

```

```

best_algorithm = 'SA';

end

fprintf('选择 %s 算法的结果作为最终策略 (适应度: %.2f)\n', best_algorithm,
max([ga_best_fitness, pso_best_fitness, sa_best_fitness]));

% 使用最佳个体生成最终策略

[drone_strategy, bomb_strategy, effective_times, missile_targets,
total_effective_time, time_intervals] = generate_strategy(best_individual,
drones, ...

target_center, g, v_smoke_sink, ...

smoke_effective_radius, smoke_effective_time, ...

missiles, missile_speed, missile_names);

% 保存结果到 Excel

save_to_excel(drone_strategy, bomb_strategy, effective_times,
missile_targets, drone_names, total_effective_time);

% 绘制收敛曲线比较

figure;

plot(1:length(ga_history), ga_history, 'b-', 'LineWidth', 1.5);

hold on;

plot(1:length(pso_history), pso_history, 'r-', 'LineWidth', 1.5);

plot(1:length(sa_history), sa_history, 'g-', 'LineWidth', 1.5);

xlabel('迭代次数');

ylabel('最佳适应度');

title('不同优化算法收敛曲线比较');

```

```

legend('遗传算法(GA)', '粒子群优化(PSO)', '模拟退火(SA)');

grid on;

hold off;

% 绘制无人机和导弹轨迹

plot_trajectories(drones, drone_strategy, missiles, missile_speed,
target_center);

% 输出最终遮蔽时间

fprintf('\n 最终遮蔽时间分析:\n');

fprintf('总有效遮蔽时间: %.2f 秒\n', total_effective_time);

fprintf('各干扰弹有效遮蔽时间:\n');

for i = 1:length(effective_times)

drone_idx = ceil(i/3);

bomb_idx = mod(i-1, 3) + 1;

fprintf(' %s 的第%d 枚干扰弹: %.2f 秒\n', drone_names{drone_idx}, bomb_idx,
effective_times(i));

end

% 绘制遮蔽时间区间图

plot_time_intervals(time_intervals, total_effective_time);

%% 遗传算法实现

function [best_individual, best_fitness, history] = run_ga(drones,
target_center, ...

smoke_effective_radius, g, v_smoke_sink, ...

```

```

smoke_effective_time, missiles, missile_speed, ...

pop_size, max_gen)

% 决策变量范围

min_speed = 70;

max_speed = 140;

min_heading = 0;

max_heading = 2*pi;

min_drop_time = 0;

max_drop_time = 70;

min_tau = 0;

max_tau = 20;

mutation_rate = 0.1;

% 初始化种群

population = initialize_population(pop_size, drones, min_speed, max_speed, ...

min_heading, max_heading, min_drop_time, ...

max_drop_time, min_tau, max_tau);

% 遗传算法主循环

best_fitness = 0;

best_individual = [];

history = zeros(max_gen, 1);

for gen = 1:max_gen

% 评估适应度

fitness = evaluate_population(population, drones, target_center, ...

smoke_effective_radius, g, v_smoke_sink, ...

smoke_effective_time, missiles, missile_speed);

% 记录最佳个体

```

```

[max_fit, idx] = max(fitness);

history(gen) = max_fit;

if max_fit > best_fitness

best_fitness = max_fit;

best_individual = population(idx,:);

end

% 选择

selected = selection(population, fitness);

% 交叉

offspring = crossover(selected);

% 变异

offspring = mutation(offspring, mutation_rate, min_speed, max_speed, ...
min_heading, max_heading, min_drop_time, max_drop_time, ...
min_tau, max_tau);

% 更新种群

population = offspring;

% 显示进度

if mod(gen, 10) == 0

fprintf('GA 第 %d 代, 最佳适应度: %.2f\n', gen, max_fit);

end

end

end

%% 粒子群优化实现

function [best_individual, best_fitness, history] = run_pso(drones,
target_center, ...

```



```

smoke_effective_radius, g, v_smoke_sink, ...

smoke_effective_time, missiles, missile_speed, ...

pop_size, max_gen)

% 决策变量范围

min_speed = 70;

max_speed = 140;

min_heading = 0;

max_heading = 2*pi;

min_drop_time = 0;

max_drop_time = 70;

min_tau = 0;

max_tau = 20;

n_drones = size(drones, 1);

n_vars = n_drones * 8; % 每个无人机有 8 个参数

% PSO 参数

w = 0.7; % 惯性权重

c1 = 1.5; % 个体学习因子

c2 = 1.5; % 社会学习因子

% 初始化粒子群

particles = zeros(pop_size, n_vars);

velocities = zeros(pop_size, n_vars);

pbest = zeros(pop_size, n_vars);

pbest_fitness = zeros(pop_size, 1);

for i = 1:pop_size

particles(i, :) = initialize_individual(drones, min_speed, max_speed, ...

min_heading, max_heading, min_drop_time, ...

```

```

max_drop_time, min_tau, max_tau);

velocities(i, :) = zeros(1, n_vars);

pbest(i, :) = particles(i, :);

pbest_fitness(i) = evaluate_individual(particles(i, :), drones,
target_center, ...

smoke_effective_radius, g, v_smoke_sink, ...

smoke_effective_time, missiles, missile_speed);

end

% 全局最佳

[gbest_fitness, gbest_idx] = max(pbest_fitness);

gbest = pbest(gbest_idx, :);

% PSO 主循环

best_fitness = gbest_fitness;

best_individual = gbest;

history = zeros(max_gen, 1);

for gen = 1:max_gen

for i = 1:pop_size

% 更新速度

r1 = rand(1, n_vars);

r2 = rand(1, n_vars);

velocities(i, :) = w * velocities(i, :) + ...

c1 * r1 .* (pbest(i, :) - particles(i, :)) + ...

c2 * r2 .* (gbest - particles(i, :));

% 更新位置

particles(i, :) = particles(i, :) + velocities(i, :);

% 边界处理

```

```

particles(i, :) = bound_check(particles(i, :), drones, min_speed,
max_speed, ...

min_heading, max_heading, min_drop_time, max_drop_time, ...

min_tau, max_tau);

% 评估适应度

fitness = evaluate_individual(particles(i, :), drones, target_center, ...

smoke_effective_radius, g, v_smoke_sink, ...

smoke_effective_time, missiles, missile_speed);

% 更新个体最佳

if fitness > pbest_fitness(i)

pbest(i, :) = particles(i, :);

pbest_fitness(i) = fitness;

% 更新全局最佳

if fitness > gbest_fitness

gbest = particles(i, :);

gbest_fitness = fitness;

end

end

end

history(gen) = gbest_fitness;

if gbest_fitness > best_fitness

best_fitness = gbest_fitness;

best_individual = gbest;

end

% 显示进度

if mod(gen, 10) == 0

```

```

fprintf('PSO 第 %d 代, 最佳适应度: %.2f\n', gen, gbest_fitness);

end

end

end

%% 模拟退火实现

function [best_individual, best_fitness, history] = run_sa(drones,
target_center, ...

smoke_effective_radius, g, v_smoke_sink, ...

smoke_effective_time, missiles, missile_speed, ...

max_gen)

% 决策变量范围

min_speed = 70;

max_speed = 140;

min_heading = 0;

max_heading = 2*pi;

min_drop_time = 0;

max_drop_time = 70;

min_tau = 0;

max_tau = 20;

n_drones = size(drones, 1);

% SA 参数

T0 = 1000; % 初始温度

Tf = 1; % 最终温度

alpha = 0.95; % 降温系数

% 初始化当前解

```

```

current_solution = initialize_individual(drones, min_speed, max_speed, ...
min_heading, max_heading, min_drop_time, ...
max_drop_time, min_tau, max_tau);

current_fitness = evaluate_individual(current_solution, drones,
target_center, ...
smoke_effective_radius, g, v_smoke_sink, ...
smoke_effective_time, missiles, missile_speed);

% 最佳解
best_solution = current_solution;
best_fitness = current_fitness;

% SA 主循环
T = T0;
history = zeros(max_gen, 1);
gen = 1;
while T > Tf && gen <= max_gen
% 生成新解
new_solution = mutate_individual(current_solution, 1.0, min_speed,
max_speed, ...
min_heading, max_heading, min_drop_time, max_drop_time, ...
min_tau, max_tau);
new_fitness = evaluate_individual(new_solution, drones, target_center, ...
smoke_effective_radius, g, v_smoke_sink, ...
smoke_effective_time, missiles, missile_speed);

% 计算适应度差
delta = new_fitness - current_fitness;

% 接受新解的条件

```

```

if delta > 0 || rand() < exp(delta / T)

current_solution = new_solution;

current_fitness = new_fitness;

if current_fitness > best_fitness

best_solution = current_solution;

best_fitness = current_fitness;

end

end

% 降温

T = T * alpha;

history(gen) = best_fitness;

gen = gen + 1;

% 显示进度

if mod(gen, 10) == 0

fprintf('SA 第 %d 代, 最佳适应度: %.2f, 温度: %.2f\n', gen, best_fitness, T);

end

end

best_individual = best_solution;

end

%% 辅助函数

function pop = initialize_population(pop_size, drones, min_speed,
max_speed, ...

min_heading, max_heading, min_drop_time, ...

max_drop_time, min_tau, max_tau)

n_drones = size(drones, 1);

```

```

n_bombs_per_drone = 3;

% 每个个体的编码: [速度 1, 航向 1, 投放时间 1,  $\tau_1$ , 投放时间 2,  $\tau_2$ , 投放时间 3,
 $\tau_3$ , ...]

pop = zeros(pop_size, n_drones * (2 + n_bombs_per_drone * 2));

for i = 1:pop_size
    individual = [];

    for d = 1:n_drones

        % 无人机速度和航向

        speed = min_speed + (max_speed - min_speed) * rand();

        heading = min_heading + (max_heading - min_heading) * rand();

        % 三枚干扰弹的投放时间和 $\tau$ 

        bomb_params = [];

        for b = 1:n_bombs_per_drone

            drop_time = min_drop_time + (max_drop_time - min_drop_time) * rand();

            tau = min_tau + (max_tau - min_tau) * rand();

            bomb_params = [bomb_params, drop_time, tau];

        end

        individual = [individual, speed, heading, bomb_params];

    end

    pop(i, :) = individual;

end

end

function fitness = evaluate_population(population, drones, target_center, ...
    smoke_effective_radius, g, v_smoke_sink, ...
    smoke_effective_time, missiles, missile_speed)

```

```

pop_size = size(population, 1);

fitness = zeros(pop_size, 1);

for i = 1:pop_size
    individual = population(i, :);

    total_effective_time = 0;

    % 计算每架无人机的贡献

    for d = 1:size(drones, 1)
        drone_pos = drones(d, :);

        idx = (d-1)*8 + 1;

        speed = individual(idx);

        heading = individual(idx+1);

        % 计算无人机运动方向向量

        dx = cos(heading);

        dy = sin(heading);

        % 计算三枚干扰弹的贡献

        for b = 1:3

            drop_time = individual(idx + 1 + 2*(b-1) + 1); % idx+2, idx+4, idx+6

            tau = individual(idx + 1 + 2*(b-1) + 2); % idx+3, idx+5, idx+7

            % 计算投放点

            drop_x = drone_pos(1) + speed * dx * drop_time;

            drop_y = drone_pos(2) + speed * dy * drop_time;

            drop_z = drone_pos(3);

            % 计算起爆点

            detonation_time = drop_time + tau;

            fall_distance = 0.5 * g * tau^2;

            detonation_z = drop_z - fall_distance;

```



```

% 烟幕云团中心位置（考虑下沉）

smoke_center = [drop_x + speed * dx * tau, ...
drop_y + speed * dy * tau, ...
detonation_z];

% 计算有效遮蔽时间

effective_time = calculate_effective_time(smoke_center, target_center, ...
detonation_time, v_smoke_sink, ...
smoke_effective_radius, smoke_effective_time);

total_effective_time = total_effective_time + effective_time;

end

end

fitness(i) = total_effective_time;

end

end

function [effective_time, start_time, end_time] =
calculate_effective_time(smoke_center, target_center, ...
detonation_time, v_smoke_sink, ...
smoke_effective_radius, smoke_effective_time)

% 计算烟幕云团中心与目标中心的距离

distance = norm(smoke_center - target_center);

% 如果初始距离超过有效半径，计算何时进入有效范围

if distance > smoke_effective_radius

% 烟幕下沉过程中，中心与目标的距离会变化

% 简化模型：假设烟幕中心垂直下沉

vertical_distance = abs(smoke_center(3) - target_center(3));

```

```

% 计算进入和离开有效范围的时间

time_to_enter = max(0, (vertical_distance - smoke_effective_radius) /
v_smoke_sink);

time_in_effect = min(smoke_effective_time, 2 * smoke_effective_radius /
v_smoke_sink);

effective_time = max(0, time_in_effect - time_to_enter);

start_time = detonation_time + time_to_enter;

end_time = start_time + effective_time;

else

% 如果初始就在有效范围内

vertical_distance = abs(smoke_center(3) - target_center(3));

time_in_effect = min(smoke_effective_time, (smoke_effective_radius +
vertical_distance) / v_smoke_sink);

effective_time = time_in_effect;

start_time = detonation_time;

end_time = start_time + effective_time;

end

end

function selected = selection(population, fitness)

pop_size = size(population, 1);

% 确保适应度为正值

min_fit = min(fitness);

if min_fit < 0

fitness = fitness - min_fit + eps;

end

```

```

% 计算选择概率

total_fitness = sum(fitness);

prob = fitness / total_fitness;

% 轮盘赌选择

selected_idx = randsample(1:pop_size, pop_size, true, prob);

selected = population(selected_idx, :);

end

function offspring = crossover(selected)

pop_size = size(selected, 1);

n_vars = size(selected, 2);

offspring = zeros(size(selected));

for i = 1:2:pop_size-1

parent1 = selected(i, :);

parent2 = selected(i+1, :);

% 随机选择交叉点

cross_point = randi([1, n_vars-1]);

% 执行交叉

offspring(i, :) = [parent1(1:cross_point), parent2(cross_point+1:end)];

offspring(i+1, :) = [parent2(1:cross_point), parent1(cross_point+1:end)];

end

end

function offspring = mutation(offspring, mutation_rate, min_speed,
max_speed, ...

min_heading, max_heading, min_drop_time, max_drop_time, ...

```

```

min_tau, max_tau)

pop_size = size(offspring, 1);

n_vars = size(offspring, 2);

n_drones = 5;

for i = 1:pop_size
    if rand() < mutation_rate
        % 随机选择变异点
        mut_point = randi([1, n_vars]);

        % 根据变异点的位置确定变异范围

        drone_idx = ceil(mut_point / 8); % 确定属于哪个无人机

        param_idx = mod(mut_point - 1, 8) + 1; % 确定是哪个参数

        if param_idx == 1 % 速度参数
            offspring(i, mut_point) = min_speed + (max_speed - min_speed) * rand();

        elseif param_idx == 2 % 航向参数
            offspring(i, mut_point) = min_heading + (max_heading - min_heading) * rand();

        elseif ismember(param_idx, [3, 5, 7]) % 投放时间
            offspring(i, mut_point) = min_drop_time + (max_drop_time - min_drop_time) *
            rand();

        elseif ismember(param_idx, [4, 6, 8]) %  $\tau$ 参数
            offspring(i, mut_point) = min_tau + (max_tau - min_tau) * rand();

        end

    end

end

end

function individual = initialize_individual(drones, min_speed, max_speed, ...

```

```

min_heading, max_heading, min_drop_time, ...
max_drop_time, min_tau, max_tau)

n_drones = size(drones, 1);

individual = [];

for d = 1:n_drones
% 无人机速度和航向

speed = min_speed + (max_speed - min_speed) * rand();

heading = min_heading + (max_heading - min_heading) * rand();

% 三枚干扰弹的投放时间和 $\tau$ 

bomb_params = [];

for b = 1:3

drop_time = min_drop_time + (max_drop_time - min_drop_time) * rand();

tau = min_tau + (max_tau - min_tau) * rand();

bomb_params = [bomb_params, drop_time, tau];

end

individual = [individual, speed, heading, bomb_params];

end

end

function fitness = evaluate_individual(individual, drones, target_center, ...
smoke_effective_radius, g, v_smoke_sink, ...
smoke_effective_time, missiles, missile_speed)

total_effective_time = 0;

% 计算每架无人机的贡献

for d = 1:size(drones, 1)

drone_pos = drones(d, :);

```

```

idx = (d-1)*8 + 1;

speed = individual(idx);

heading = individual(idx+1);

% 计算无人机运动方向向量

dx = cos(heading);

dy = sin(heading);

% 计算三枚干扰弹的贡献

for b = 1:3

    drop_time = individual(idx + 1 + 2*(b-1) + 1); % idx+2, idx+4, idx+6

    tau = individual(idx + 1 + 2*(b-1) + 2); % idx+3, idx+5, idx+7

% 计算投放点

    drop_x = drone_pos(1) + speed * dx * drop_time;

    drop_y = drone_pos(2) + speed * dy * drop_time;

    drop_z = drone_pos(3);

% 计算起爆点

    detonation_time = drop_time + tau;

    fall_distance = 0.5 * g * tau^2;

    detonation_z = drop_z - fall_distance;

% 烟幕云团中心位置（考虑下沉）

    smoke_center = [drop_x + speed * dx * tau, ...

        drop_y + speed * dy * tau, ...

        detonation_z];

% 计算有效遮蔽时间

    effective_time = calculate_effective_time(smoke_center, target_center, ...

        detonation_time, v_smoke_sink, ...

        smoke_effective_radius, smoke_effective_time);

```

```

total_effective_time = total_effective_time + effective_time;

end

end

fitness = total_effective_time;

end

function individual = bound_check(individual, drones, min_speed, max_speed, ...
min_heading, max_heading, min_drop_time, max_drop_time, ...
min_tau, max_tau)

n_drones = size(drones, 1);

for d = 1:n_drones

idx = (d-1)*8 + 1;

% 速度和航向边界检查

individual(idx) = max(min(individual(idx), max_speed), min_speed); % 速度

individual(idx+1) = mod(individual(idx+1), 2*pi); % 航向, 保持在 0-2 $\pi$ 范围内

% 炸弹参数边界检查

for b = 1:3

drop_time_idx = idx + 1 + 2*(b-1) + 1;

tau_idx = idx + 1 + 2*(b-1) + 2;

individual(drop_time_idx) = max(min(individual(drop_time_idx),
max_drop_time), min_drop_time);

individual(tau_idx) = max(min(individual(tau_idx), max_tau), min_tau);

end

end

end

```

```

function mutated = mutate_individual(individual, mutation_rate, min_speed,
max_speed, ...

min_heading, max_heading, min_drop_time, max_drop_time, ...

min_tau, max_tau)

n_vars = length(individual);

n_drones = 5;

mutated = individual;

for i = 1:n_vars

if rand() < mutation_rate

% 根据变异点的位置确定变异范围

drone_idx = ceil(i / 8); % 确定属于哪个无人机

param_idx = mod(i - 1, 8) + 1; % 确定是哪个参数

if param_idx == 1 % 速度参数

mutated(i) = min_speed + (max_speed - min_speed) * rand();

elseif param_idx == 2 % 航向参数

mutated(i) = min_heading + (max_heading - min_heading) * rand();

elseif ismember(param_idx, [3, 5, 7]) % 投放时间

mutated(i) = min_drop_time + (max_drop_time - min_drop_time) * rand();

elseif ismember(param_idx, [4, 6, 8]) %  $\tau$ 参数

mutated(i) = min_tau + (max_tau - min_tau) * rand();

end

end

end

end

function [drone_strategy, bomb_strategy, effective_times, missile_targets,

```



```

total_effective_time, time_intervals] = generate_strategy(best_individual,
drones, ...

target_center, g, v_smoke_sink, ...

smoke_effective_radius, smoke_effective_time, ...

missiles, missile_speed, missile_names)

n_drones = size(drones, 1);

drone_strategy = cell(n_drones, 3); % 无人机编号, 速度, 航向角(度)

bomb_strategy = cell(n_drones*3, 10); % 无人机编号, 弹序号, 投放时间, 投放点
x,y,z, 起爆时间, 起爆点 x,y,z

effective_times = zeros(n_drones*3, 1);

missile_targets = cell(n_drones*3, 1);

time_intervals = cell(n_drones*3, 3); % 存储每个干扰弹的开始时间、结束时间和持续
时间

for d = 1:n_drones
    drone_pos = drones(d, :);

    idx = (d-1)*8 + 1;

    speed = best_individual(idx);

    heading = best_individual(idx+1);

    heading_deg = rad2deg(heading);

    if heading_deg < 0
        heading_deg = heading_deg + 360;
    end

    % 存储无人机策略

    drone_strategy{d, 1} = sprintf('FY%d', d);

    drone_strategy{d, 2} = heading_deg;

    drone_strategy{d, 3} = speed;

```

```

% 计算无人机运动方向向量
dx = cos(heading);
dy = sin(heading);

% 计算三枚干扰弹的参数
for b = 1:3
    drop_time = best_individual(idx + 1 + 2*(b-1) + 1); % idx+2, idx+4, idx+6
    tau = best_individual(idx + 1 + 2*(b-1) + 2); % idx+3, idx+5, idx+7

% 计算投放点
drop_x = drone_pos(1) + speed * dx * drop_time;
drop_y = drone_pos(2) + speed * dy * drop_time;
drop_z = drone_pos(3);

% 计算起爆点
detonation_time = drop_time + tau;
fall_distance = 0.5 * g * tau^2;
detonation_z = drop_z - fall_distance;
detonation_x = drop_x + speed * dx * tau;
detonation_y = drop_y + speed * dy * tau;

% 计算烟幕中心位置
smoke_center = [detonation_x, detonation_y, detonation_z];

% 计算有效遮蔽时间和时间区间
[effective_time, start_time, end_time] =
calculate_effective_time(smoke_center, target_center, ...
detonation_time, v_smoke_sink, ...
smoke_effective_radius, smoke_effective_time);

% 确定干扰的导弹
interfered_missiles = determine_interfered_missiles(smoke_center,

```

```

detonation_time, ...

effective_time, missiles, missile_speed, ...

target_center, missile_names, smoke_effective_radius);

% 存储炸弹策略

row_idx = (d-1)*3 + b;

bomb_strategy{row_idx, 1} = sprintf('FY%d', d);

bomb_strategy{row_idx, 2} = b;

bomb_strategy{row_idx, 3} = drop_time;

bomb_strategy{row_idx, 4} = drop_x;

bomb_strategy{row_idx, 5} = drop_y;

bomb_strategy{row_idx, 6} = drop_z;

bomb_strategy{row_idx, 7} = detonation_time;

bomb_strategy{row_idx, 8} = detonation_x;

bomb_strategy{row_idx, 9} = detonation_y;

bomb_strategy{row_idx, 10} = detonation_z;

effective_times(row_idx) = effective_time;

missile_targets{row_idx} = interfered_missiles;

% 存储时间区间信息

time_intervals{row_idx, 1} = start_time;

time_intervals{row_idx, 2} = end_time;

time_intervals{row_idx, 3} = effective_time;

end

end

% 计算总的有效遮蔽时间（考虑时间重叠）

total_effective_time = calculate_total_effective_time(time_intervals);

end

```

```

function total_time = calculate_total_effective_time(time_intervals)

% 将所有时间区间合并，计算总的有效遮蔽时间（考虑重叠）

n_intervals = size(time_intervals, 1);

intervals = zeros(n_intervals, 2);

for i = 1:n_intervals

intervals(i, 1) = time_intervals{i, 1};

intervals(i, 2) = time_intervals{i, 2};

end

% 按开始时间排序

[~, idx] = sort(intervals(:, 1));

intervals = intervals(idx, :);

% 合并重叠区间

merged = [];

current_start = intervals(1, 1);

current_end = intervals(1, 2);

for i = 2:n_intervals

if intervals(i, 1) <= current_end

% 区间重叠，扩展当前区间

current_end = max(current_end, intervals(i, 2));

else

% 区间不重叠，保存当前区间并开始新区间

merged = [merged; current_start, current_end];

current_start = intervals(i, 1);

current_end = intervals(i, 2);

end

```

```

end

merged = [merged; current_start, current_end];

% 计算总时间

total_time = 0;

for i = 1:size(merged, 1)

total_time = total_time + (merged(i, 2) - merged(i, 1));

end

end

function interfered_missiles = determine_interfered_missiles(smoke_center,
detonation_time, ...

effective_time, missiles, missile_speed, ...

target_center, missile_names, smoke_effective_radius)

n_missiles = size(missiles, 1);

interfered_missiles = '';

for m = 1:n_missiles

missile_pos = missiles(m, :);

missile_dir = (target_center - missile_pos) / norm(target_center -
missile_pos);

% 计算导弹到达目标的时间

distance_to_target = norm(target_center - missile_pos);

arrival_time = distance_to_target / missile_speed;

% 计算导弹在烟幕起爆时的位置

missile_pos_at_detonation = missile_pos + missile_dir * missile_speed *
detonation_time;

% 计算导弹与烟幕中心的距离

```

```

distance_to_smoke = norm(missile_pos_at_detonation - smoke_center);

% 如果距离小于烟幕有效半径，则认为干扰了该导弹

if distance_to_smoke <= smoke_effective_radius

if ~isempty(interfered_missiles)

interfered_missiles = [interfered_missiles, ','];

end

interfered_missiles = [interfered_missiles, missile_names{m}];

end

end

if isempty(interfered_missiles)

interfered_missiles = '无';

end

end

function save_to_excel(drone_strategy, bomb_strategy, effective_times,
missile_targets, drone_names, total_effective_time)

% 创建结果表格

results = cell(19, 12); % 19 行: 15 个炸弹 + 1 个表头 + 3 个备注行

% 表头

headers = {'无人机运动方向', '无人机运动速度 (m/s)', '烟幕干扰弹编号', ...
'烟幕干扰弹投放点的 x 坐标 (m)', '烟幕干扰弹投放点的 y 坐标 (m)', '烟幕干扰弹投放点的 z 坐标 (m)', ...
'烟幕干扰弹起爆点的 x 坐标 (m)', '烟幕干扰弹起爆点的 y 坐标 (m)', '烟幕干扰弹起爆点的 z 坐标 (m)', ...
'有效干扰时长 (s)', '干扰的导弹编号'};

for i = 1:length(headers)

```

```

results{1, i} = headers{i};

end

% 填充数据

row = 2;

for d = 1:5

    drone_name = drone_names{d};

    drone_dir = drone_strategy{d, 2};

    drone_speed = drone_strategy{d, 3};

    for b = 1:3

        idx = (d-1)*3 + b;

        results{row, 1} = drone_dir;

        results{row, 2} = drone_speed;

        results{row, 3} = b;

        results{row, 4} = bomb_strategy{idx, 4};

        results{row, 5} = bomb_strategy{idx, 5};

        results{row, 6} = bomb_strategy{idx, 6};

        results{row, 7} = bomb_strategy{idx, 8};

        results{row, 8} = bomb_strategy{idx, 9};

        results{row, 9} = bomb_strategy{idx, 10};

        results{row, 10} = effective_times(idx);

        results{row, 11} = missile_targets{idx};

        row = row + 1;

    end

end

% 添加总遮蔽时间和备注

results{17, 1} = '总有效遮蔽时间';

```

```

results{17, 2} = total_effective_time;

results{17, 3} = '秒';

results{18, 1} = '注：以 x 轴为正向，逆时针方向为正，取值 0~360（度）。';

results{19, 1} = '注：总有效遮蔽时间已考虑时间重叠，是实际遮蔽时间。';

% 尝试保存到不同位置或不同文件名

filename = '烟幕干扰弹投放策略结果.xlsx';

attempts = 0;

max_attempts = 5;

saved = false;

while ~saved && attempts < max_attempts

try

% 检查文件是否存在，如果存在则尝试删除

if exist(filename, 'file')

delete(filename);

end

% 写入 Excel 文件

writecell(results, filename, 'Sheet', 'Sheet1');

saved = true;

disp(['策略已保存到 ' filename]);

catch ME

attempts = attempts + 1;

if attempts < max_attempts

% 尝试使用不同的文件名

filename = ['烟幕干扰弹投放策略结果_' datestr(now, 'yyyy-mm-dd_HH-MM-SS')

'.xlsx'];

disp(['尝试保存到 ' filename]);

```



```

pause(1); % 等待一秒再试

else

% 如果所有尝试都失败，显示错误信息

disp('无法保存 Excel 文件，可能的原因: ');

disp('1. 文件被其他程序打开');

disp('2. 没有写入权限');

disp('3. 磁盘空间不足');

disp('错误信息: ');

disp(ME.message);

% 将结果保存为 CSV 文件作为备用

csv_filename = '烟幕干扰弹投放策略结果_backup.csv';

writecell(results, csv_filename);

disp(['已将结果备份保存到 ' csv_filename]);

end

end

end

end

function plot_trajectories(drones, drone_strategy, missiles, missile_speed,
target_center)

figure;

hold on;

grid on;

% 绘制目标

plot3(target_center(1), target_center(2), target_center(3), 'ro',
'MarkerSize', 10, 'MarkerFaceColor', 'r');

```

```

text(target_center(1), target_center(2), target_center(3), '目标',
'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'right');

% 绘制导弹轨迹

colors = ['r', 'g', 'b'];

for m = 1:size(missiles, 1)

missile_pos = missiles(m, :);

missile_dir = (target_center - missile_pos) / norm(target_center -
missile_pos);

% 计算导弹到达目标的时间

distance_to_target = norm(target_center - missile_pos);

arrival_time = distance_to_target / missile_speed;

% 生成轨迹点

t = linspace(0, arrival_time, 100);

trajectory = zeros(length(t), 3);

for i = 1:length(t)

trajectory(i, :) = missile_pos + missile_dir * missile_speed * t(i);

end

% 绘制轨迹

plot3(trajectory(:, 1), trajectory(:, 2), trajectory(:, 3), [colors(m), '-'],
'LineWidth', 1.5);

plot3(missile_pos(1), missile_pos(2), missile_pos(3), [colors(m), 'o'],
'MarkerSize', 8, 'MarkerFaceColor', colors(m));

text(missile_pos(1), missile_pos(2), missile_pos(3), sprintf('M%d', m),
'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'right');

end

% 绘制无人机轨迹和干扰弹

for d = 1:5

```

```

drone_pos = drones(d, :);

drone_dir_deg = drone_strategy{d, 2};

drone_speed = drone_strategy{d, 3};

drone_dir_rad = deg2rad(drone_dir_deg);

dx = cos(drone_dir_rad);

dy = sin(drone_dir_rad);

% 绘制无人机初始位置

plot3(drone_pos(1), drone_pos(2), drone_pos(3), 'ko', 'MarkerSize', 8,
'MarkerFaceColor', 'k');

text(drone_pos(1), drone_pos(2), drone_pos(3), sprintf('FY%d', d),
'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'right');

% 绘制无人机轨迹

max_time = 70;

t = linspace(0, max_time, 100);

trajectory = zeros(length(t), 3);

for i = 1:length(t)

trajectory(i, 1) = drone_pos(1) + drone_speed * dx * t(i);

trajectory(i, 2) = drone_pos(2) + drone_speed * dy * t(i);

trajectory(i, 3) = drone_pos(3);

end

plot3(trajectory(:, 1), trajectory(:, 2), trajectory(:, 3), 'k--', 'LineWidth',
1);

end

xlabel('X (m)');

ylabel('Y (m)');

zlabel('Z (m)');

```

```

title('无人机和导弹轨迹');

legend('目标', 'M1 轨迹', 'M1', 'M2 轨迹', 'M2', 'M3 轨迹', 'M3', '无人机',
'Location', 'best');

view(3);

hold off;

end

function plot_time_intervals(time_intervals, total_effective_time)

% 绘制时间区间图

figure;

hold on;

n_intervals = size(time_intervals, 1);

colors = lines(n_intervals);

for i = 1:n_intervals

start_time = time_intervals{i, 1};

end_time = time_intervals{i, 2};

duration = time_intervals{i, 3};

% 绘制时间区间

plot([start_time, end_time], [i, i], 'LineWidth', 10, 'Color', colors(i, :));

text(start_time, i, sprintf('%.1fs', duration), 'VerticalAlignment', 'bottom',
'FontSize', 8);

end

yticks(1:n_intervals);

yticklabels(arrayfun(@(x) sprintf('干扰弹%d', x), 1:n_intervals,
'UniformOutput', false));

xlabel('时间 (秒)');

```

```
ylabel('干扰弹');  
  
title(sprintf('各干扰弹有效遮蔽时间区间 (总时间: %.2f 秒)',  
total_effective_time));  
  
grid on;  
  
hold off;  
  
end
```